

Introducción al Procesamiento de Lenguaje Natural

Cap. 1. Acercamiento a los word embeddings



Cuestiones teóricas



¿Qué es el significado de una palabra?

- Hasta ahora en los modelos que trabajamos (n-gramas) no había una forma de definir la semántica: las palabras (o tokens) eran simplemente strings
- Veamos algunas ideas asociadas a la noción de significado (proviene de la léxicosemántica)
- ¿Cómo representar el significado de una palabra?
 - **Lemma:** a forma base o estandarizada de una palabra, tal como aparecería en la entrada de un diccionario. Es la forma que representa a todas las variantes de una palabra.
 - **Sentido:** se refiere a cada uno de los distintos significados que puede tener un mismo lemma



- Banco, bancos, bancada => mismo lemma
- Un mismo lemma puede tener muchos **sentidos** => polisemia

Artículo

Sinónimos o afines

Del fr. ant. *bank*, y este del germ. **banki*.

1. m. Asiento, con respaldo o sin él, en que pueden sentarse dos o más personas.
Sin.: banca, asiento, banqueta, taburete, grada¹, silla, poyo.
2. m. Madero grueso escuadrado que se coloca horizontalmente sobre cuatro pies y sirve de mesa para labores de carpinteros y otros artesanos.
3. m. En los mares, ríos y lagos navegables, bajo que se prolonga en una gran extensión.
Sin.: bajo, bajo, alfaque.
4. m. Conjunto de peces que van juntos en gran número.
Sin.: bando², cardumen, manjúa.
5. m. Empresa dedicada a realizar operaciones financieras con el dinero procedente de sus accionistas y de los depósitos de sus clientes.
6. m. Establecimiento médico donde se conservan y almacenan órganos, tejidos o líquidos fisiológicos humanos para cubrir necesidades quirúrgicas, de investigación, etc. Banco de ojos, de sangre.
7. m. Arq. sotabanco (II piso habitable).
Sin.: sotabanco.
8. m. Arq. Base o parte inferior de un retablo, que puede estar dividido en dos pisos.
Sin.: predela.
9. m. Geol. Estrato de gran espesor.
10. m. Ingen. Macizo de mineral que presenta dos caras descubiertas, una horizontal superior y otra vertical.
Sin.: antepecho.
11. m. Ven. Extensión de terreno con vegetación arbórea que sobresale en la llanura.



Relaciones entre palabras (relaciones léxicas)

- Son las diversas formas en que los sentidos de las palabras se conectan entre sí, constituyendo un componente fundamental del significado léxico
- Se trata de ver a las palabras no de forma aislada, sino cómo se vinculan mediante similitudes, oposiciones o asociaciones contextuales

Relaciones léxicas: sinonimia

- Dos palabras tienen un sentido idéntico:
 - Automóvil / coche
 - Sofá / sillón
 - Agua / H₂O
 - Oculista / Oftalmólogo
- Definición más técnica: dos palabras son sinónimas si son sustituibles entre sí en cualquier oración sin cambiar las condiciones de verdad de la misma, es decir, las situaciones en las que la oración sería verdadera.



Relaciones léxicas: sinonimia

- **Principio de contraste:** casi no existen palabras cuyo significado es exactamente el mismo. En general, las diferencias en las formas de escritura llevan implícitas diferencias en el sentido o en uso (jergas, dominios, etc.)
 - Oculista / Oftalmólogo: diferencia de contextos de uso (+ informal / + formal)
 - Agua / H₂O: difícilmente dos nadadores vayan a decir “voy a meterme al H₂O”



Similaridad entre palabras

- Si bien no parecen existir sinónimos estrictos, las palabras sí tienen muchas palabras *similares*.
- La *similaridad* agrupa palabras que, aunque no significan lo mismo, comparten muchas características.
 - Por ejemplo, *gato* y *perro* no son sinónimos, pero son palabras muy similares
- En NLP => pasar de la noción de sinonimia a la de similaridad y dejar de hablar de relaciones entre lemmas y pasar a hablar de relaciones entre palabras.
- Trabajar con relaciones entre palabras completas en lugar de tener que comprometernos con una representación específica de los sentidos de las palabras. Eviamos tener que definir cada sentido individual.



Similaridad entre palabras

- Podemos asociar similitud entre dos palabras con similitud de significado => si tenemos alguna métrica de cuán similares son dos palabras, podemos intentar cuantificar cuán similares son sus sentidos => importante en question answering, summarization, etc.
- ¿Cómo?
 - Preguntarle la gente qué tan similares son dos palabras

vanish	disappear	9.8
belief	impression	5.95
muscle	bone	3.65
modest	flexible	0.98
hole	agreement	0.3

SimLex-999 dataset (Hill et al., 2015)

Asociación entre palabras

- Hay otras formas de relación entre palabras: asociación entre palabras (*word relatedness* o *word association*).
- Ejemplo: “café” / “taza”
- No son similares: el café es un grano que se procesa para generar una bebida que se llama igual; una taza es un objeto manufacturado con una forma definida para contener líquidos.



- Pero tiene una *relación* (o *asociación*). Ambas comparten un evento cotidiano: desayunos o meriendas o eventos en los que se toma café.
- En cambio, “té” y “café” sí son similares en el sentido anterior



Campos semánticos

- Conjunto de palabras que cubren un dominio específico y mantienen relaciones estructuradas entre sí.
 - *Hospitales*: cirujanos, enfermeras, escalpelos, camas, anestesia, etc...
 - *Muebles*: lámpara, mesa de luz, mesa ratona, biblioteca, etc.
 - *Deportes*: fútbol, pelota, rugby, arco, jugadores, equipos
- Podemos pensar que los modelos de tópicos son una forma de “operacionalizar” la idea de campo semántico.



Relaciones: antinomia

- Sentidos que son opuestos en relación a una sola de las dimensiones del significado:
 - alto/ bajo; corto/ largo; lento/ rápido; ascenso/ descenso
 - arriba/ abajo; frío/ caliente; dentro/ fuera
- Pueden
 - definir una oposición binaria o representar puntos extremos de una escala
 - largo / corto; rápido/ lento
 - ser “reversivos”:
 - arriba / abajo



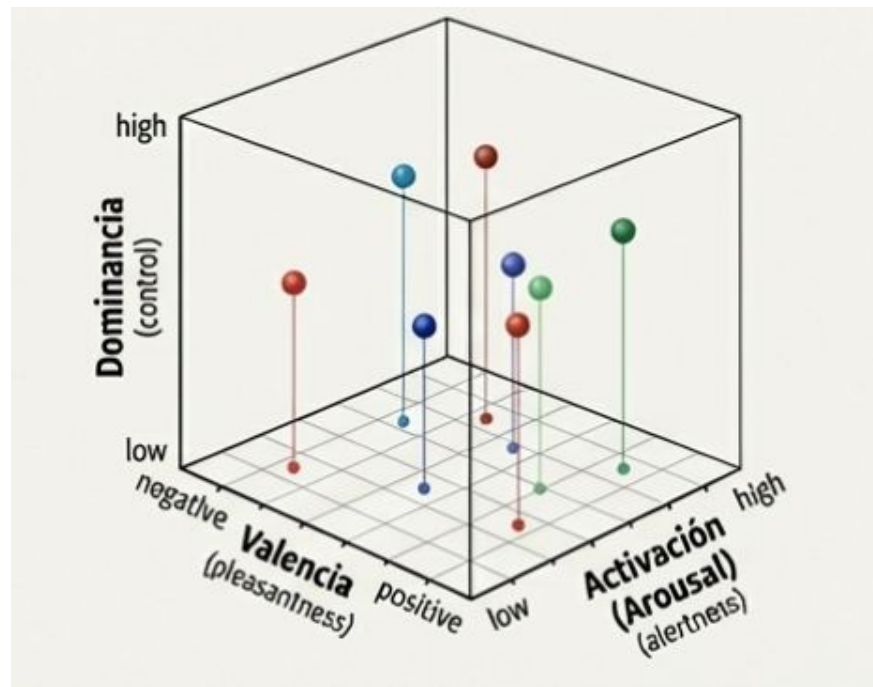
Relaciones: connotación

- Aspectos afectivos del significado, relacionados con las emociones, sentimientos u opiniones que evoca una palabra.
- Diferentes usos en diferentes disciplinas: aquí nos referimos a aquellos aspectos del significado de una palabra asociados a ciertas características del emisor (emociones, opiniones, sentimientos o evaluaciones).
 - Lo más evidente: “feliz” / “triste”
 - Más sutiles: “réplica” / “falsificación”
 - Evaluación (sentiment): “grandioso”, “amor” / “terrible”, “odio”.



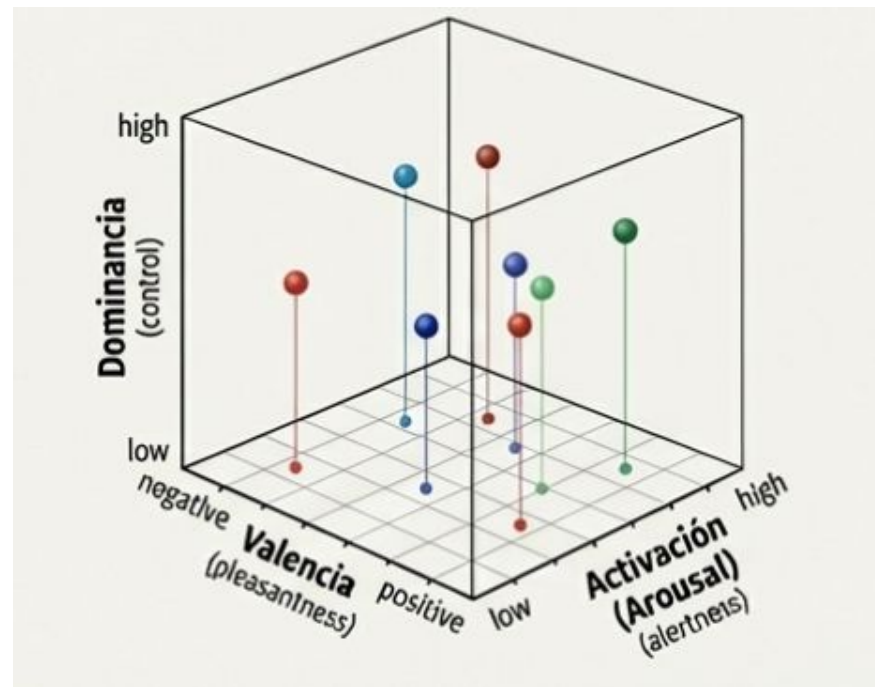
Relaciones: connotación

- Estudios parecen mostrar que muchas palabras varían en tres dimensiones:
 - Valencia (Valence): Grado de agrado o placer provocado por el estímulo. El "sentimiento" se mapea generalmente a esta dimensión.
 - Activación (Arousal): Nivel de "alerta, actividad o energía" provocado por el estímulo.
 - Dominancia (Dominance): Grado de "control o influencia" ejercido por el estímulo o la emoción.



Relaciones: connotación

- Usan un espacio de 3 dimensiones continuas para representar el significado de una palabra.
- Esta idea fue bastante disruptiva: sostiene que el sentido de una palabra puede ser expresado como un punto o como un vector en un espacio.
- Está en el origen de la “vector semantics” y de técnicas más modernas de hoy.



Semántica vectorial



¿Qué es?

- ¿Podemos generar un método o modelo que se aproxime a algunas de las relaciones que mencionamos antes?
- Semántica vectorial: es el método estándar para representar el significado de las palabras en NLP.
- En estos modelos, una palabra no se trata como una simple cadena de letras, sino como un vector, es decir, un punto en un espacio semántico multidimensional (embedding).



Ejemplo

- Tenemos estas tres oraciones
 - Mezclé **vermiculita** con la tierra para mejorar el drenaje.
 - La **vermiculita** retiene la humedad sin pudrir las raíces.
 - Usé **vermiculita** en las macetas de las suculentas.
- Y también hemos visto estas otras:
 - La perlita mezclada con tierra mejora el drenaje de las macetas.
 - La arena retiene menos humedad que otros sustratos.
 - Las raíces de las suculentas necesitan materiales sueltos y porosos.



¿Qué es la “vermiculita”?

- Es un material poroso usado en sustratos, similar a la perlita o la arena
- La inferencia es posible porque "tierra", "drenaje", "humedad", "raíces", "macetas" y "suculentas" aparecen en ambos grupos de oraciones



Hipótesis distribucional

- “El significado de una palabra es su uso en el lenguaje [dentro de un juego del lenguaje]” (Ludwig Wittgenstein)
- ¿Cómo definimos el “uso”? Una forma: las palabras están definidas por sus “entornos” o “contextos”: las palabras que están cerca.
 - “Si A y B tienen contextos casi idénticos, diremos que son sinónimos” (Zelig Harris)
 - “Conocerás una palabra por la compañía que tiene” (Robert Firth)
- Palabras cercanas tienen sentidos “cercanos”
- Idea de co-ocurrencia => términos que ocurren juntos



Ideas centrales de la semántica vectorial

1. Se define (o se “operacionaliza”, más bien) el significado de una palabra a partir de su distribución en el uso del lenguaje, es decir, sus palabras vecinas y/o su contexto gramatical.
2. El significado es un punto (o vector) en un espacio multidimensional.
 - a. Cada palabra es un vector
 - b. Las palabras cercanas (o similares) en el espacio son palabras con significados parecidos
 - c. El espacio semántico se construye a partir de las palabras que son vecinas en el corpus



¿Por qué vectores?

“Sobre la mesa hay un florero con margaritas y jazmines”

“El vaso lleno de flores está apoyado sobre una mesada”

- Mismo sentido pero ninguna palabra en común
- Una solución ya la vimos: LDA, STM
=> detección de tópicos
- Otra solución: word embeddings

	Palabra 1	Palabra 2	Palabra 3	Palabra 4	Palabra 5	
Relato 1	0	0.12	0.01	0	0	
Relato 2	0	0	0.44	0.15	0.65	
Relato 3	0.11	0.31	0.28	0	0	(...)
Relato 4	0	0	0.05	0.21	0	
Relato 5	0	0.13	0	0.07	0	
			(...)			

La correlación lineal entre filas nos da una idea de la similitud del significado entre relatos

La correlación lineal entre columnas nos da una idea de la similitud del significado entre palabras

Pero hay un problema: la mayor parte de los valores son 0

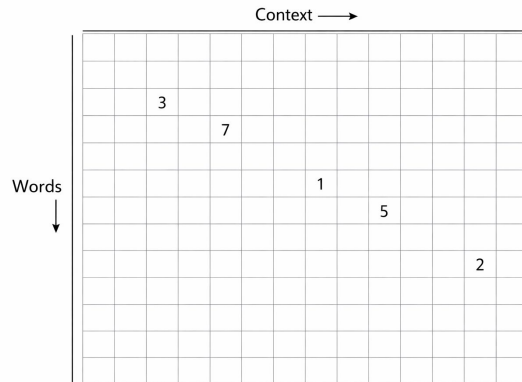
¿Por qué vectores?

- Pensemos en análisis de sentimiento
 - Usando los métodos viejos (lexicones) un feature del modelo es una identidad: tenemos que buscar las palabras exactas.
 - Usando embeddings un feature ahora es un vector y entonces podemos buscar palabras similares (con vectores similares)



Dos tipos de embeddings

- Basados en conteo
 - Matrices ralas (sparse)
 - La palabra está representada por el conteo de palabras cercanas
 - Ponderación TF-IDF
- Word embeddings
 - Matrices densas
 - Creados mediante el entrenamiento de un modelo para predecir si una palabra puede ser contexto de otra(s)
 - Extensión en embeddings contextualizados





Search by
cat word

neighbors 100

distance COSINE EUCLIDEAN

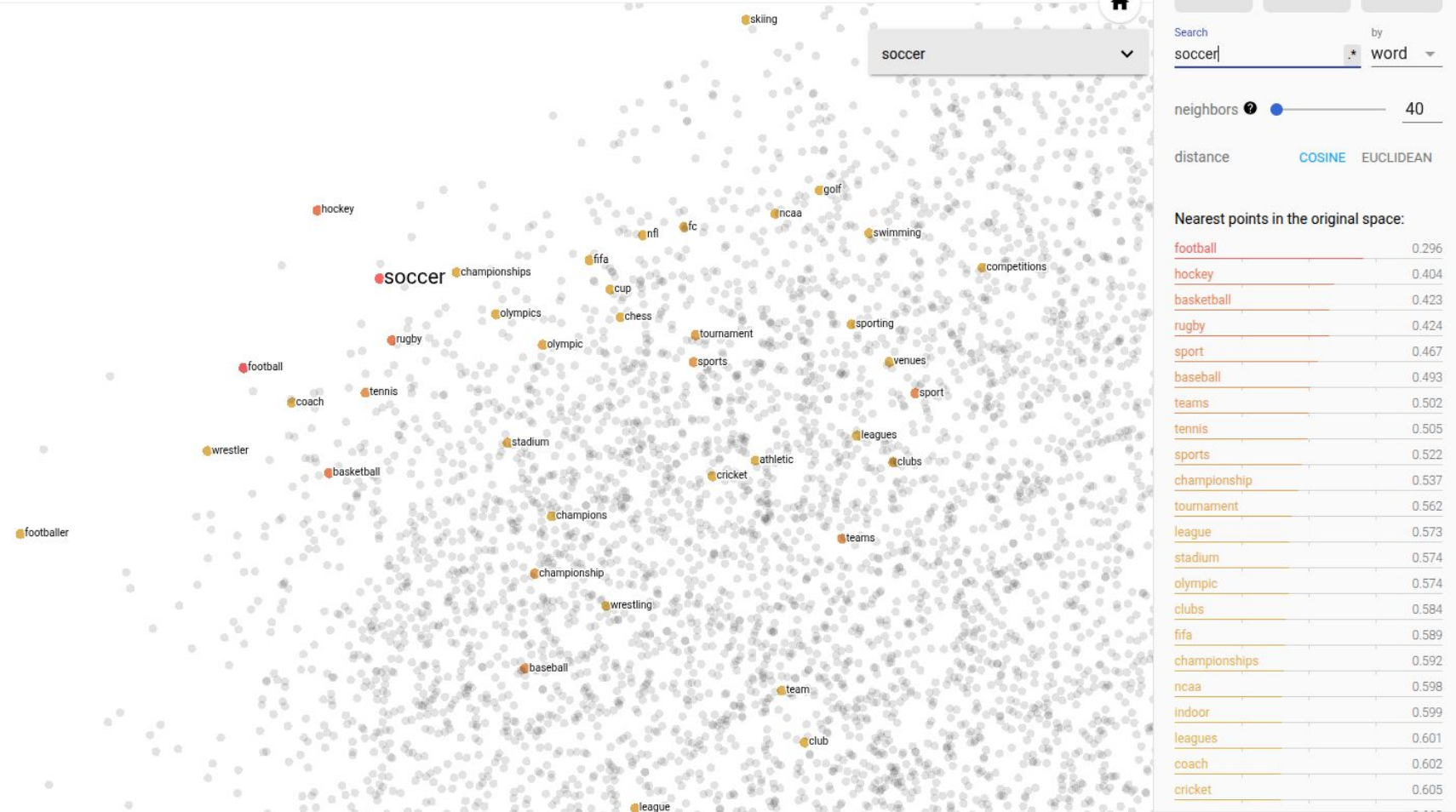
Nearest points in the original space:

protestant	0.337
church	0.370
catholicism	0.372
catholics	0.399
anglican	0.421
roman	0.423
christian	0.461
churches	0.477
orthodox	0.478
archbishop	0.515
lutheran	0.531
protestantism	0.533
protestants	0.541
bishops	0.549
christianity	0.554
episcopal	0.557
reformation	0.564
saints	0.569
bishop	0.571
cardinal	0.572
religious	0.578
baptist	0.585



factor-data
EIDAEs_UNSAM

<https://projector.tensorflow.org/>



Intuición: palabras con “entornos similares” tienen semánticas similares

- Cómo medir el contexto de una palabra?
- Matriz de co-ocurrencia de palabra-contexto
 - Cada fila representa una palabra (target) del vocabulario
 - Cada columna representa una palabra del contexto (context)
 - Cada celda representa con cuánta frecuencia una palabra del contexto aparece **cerca** del target



Embeddings basados en conteos



¿Qué quiere decir “cerca”?



WIKIPEDIA
The Free Encyclopedia

sliding window (size=2)

Choripan. The Argentine *choripán* consists of a sausage made out of beef and pork, hot off the grill, split down the middle, and served on a roll. The *chorizo* may be used whole or cut in half lengthwise, in which case it is called a *mariposa* (butterfly). It is customary to add sauces on the bread, most likely *chimichurri*.

¿Qué quiere decir “cerca”?



WIKIPEDIA
The Free Encyclopedia

sliding window (size=2)

Choripan. The Argentine *choripán* consists of a sausage made out of beef and pork, hot off the grill, split down the middle, and served on a roll. The *chorizo* may be used whole or cut in half lengthwise, in which case it is called a *mariposa* (butterfly). It is customary to add sauces on the bread, most likely *chimichurri*.

¿Qué quiere decir “cerca”?



WIKIPEDIA
The Free Encyclopedia

sliding window (size=2)

Choripan. The Argentine *choripán* consists of a sausage made out of beef and pork, hot off the grill, split down the middle, and served on a roll. The *chorizo* may be used whole or cut in half lengthwise, in which case it is called a *mariposa* (butterfly). It is customary to add sauces on the bread, most likely *chimichurri*.

¿Qué quiere decir “cerca”?



WIKIPEDIA
The Free Encyclopedia

sliding window (size=2)

Choripan. The Argentine *choripán* consists of a sausage made out of beef and pork, hot off the grill, split down the middle, and served on a roll. The *chorizo* may be used whole or cut in half lengthwise, in which case it is called a *mariposa* (butterfly). It is customary to add sauces on the bread, most likely *chimichurri*.

¿Qué quiere decir “cerca”?



WIKIPEDIA
The Free Encyclopedia

sliding window (size=2)

Choripan. The Argentine *choripán* consists of a sausage made out of beef and pork, hot off the grill, split down the middle, and served on a roll. The *chorizo* may be used whole or cut in half lengthwise, in which case it is called a *mariposa* (butterfly). It is customary to add sauces on the bread, most likely *chimichurri*.

¿Qué quiere decir “cerca”?



WIKIPEDIA
The Free Encyclopedia

sliding window (size=2)

Choripan. The Argentine *choripán* consists of a sausage made out of beef and pork, hot off the grill, split down the middle, and served on a roll. The *chorizo* may be used whole or cut in half lengthwise, in which case it is called a *mariposa* (butterfly). It is customary to add sauces on the bread, most likely *chimichurri*.

Term-Content Matrix

	choripan	vino	chimichurri	uva	pera	...	kiwi
choripan	1	54	23	5	2	...	1
vino	54	1	17	21	3	...	4
chimichurri	23	17	0	1	1	...	0
uva	5	21	1	0	20	...	19
pera	2	3	1	20	1	...	11
...	...	...	...	...	...	...	...
kiwi	1	4	0	19	11	...	0



Term-Content Matrix

“vino” aparece
54 veces cerca
de choripán

	choripan	vino	chimichurri	uva	pera	...	kiwi
choripan	1	54	23	5	2	...	1
vino	54	1	17	21	3	...	4
chimichurri	23	17	0	1	1	...	0
uva	5	21	1	0	20	...	19
pera	2	3	1	20	1	...	11
...	...	...	...	...	...	...	...
kiwi	1	4	0	19	11	...	0



Term-Document Matrix

- Co-ocurrencia de primer orden
- Asociaciones sintagmáticas
- Ejemplo:
mostaza - hamburguesa

Term-Content Matrix

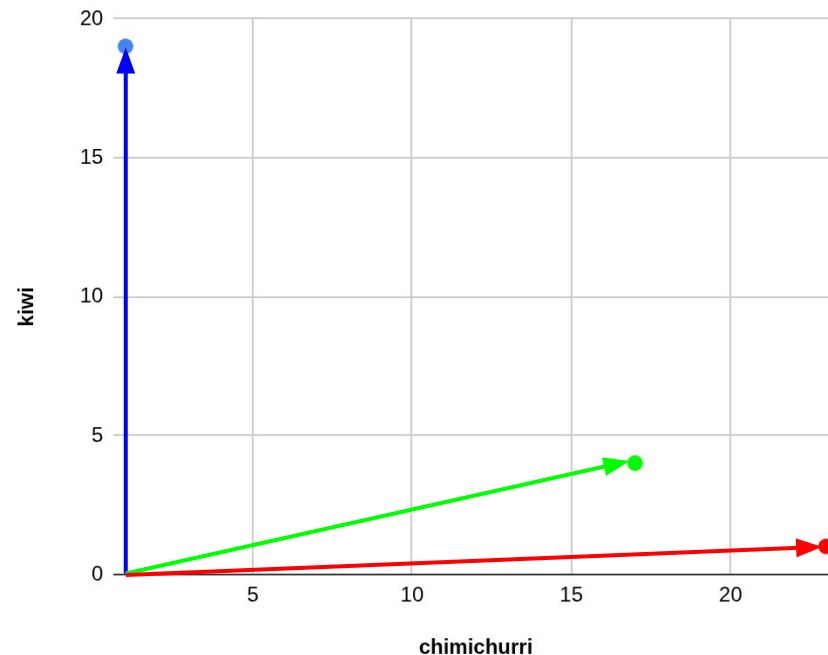
- Co-ocurrencia de segundo orden
- Asociaciones paradigmáticas
- Ejemplo:
mostaza - ketchup

Term-Content Matrix

	choripan	vino	chimichurri	uva	pera	...	kiwi
choripan	1	54	23	5	2	...	1
vino	54	1	17	21	3	...	4
chimichurri	23	17	0	1	1	...	0
uva	5	21	1	0	20	...	19
pera	2	3	1	20	1	...	11
...	...	...	...	...	...	...	...
kiwi	1	4	0	19	11	...	0

Term-Content Matrix (un subset)

	chimichurri	kiwi
choripan	23	1
vino	17	4
uva	1	19



Term-Content Matrix

- El tamaño de la Term-Context Matrix es $|V| \times |V|$
- Podría ser, en un caso no demasiado extremo, 40.000 x 40.000
- La mayoría de las celdas serán 0. ¿Por qué?
- Vectores raros o sparse



Cálculo de similaridad de palabras: similitud coseno

- El dot-product de dos vectores es un escalar

$$\text{dot product}(\mathbf{v}, \mathbf{w}) = \mathbf{v} \cdot \mathbf{w} = \sum_{i=1}^N v_i w_i = v_1 w_1 + v_2 w_2 + \dots + v_N w_N$$

- El dot product va a dar alto cuando los dos vectores tengan valores altos en las mismas direcciones
- => útil como métrica de similitud entre vectores

Cálculo de similaridad de palabras: similitud coseno

- **Problema:** el dot product tiende a favorecer vectores de mayor longitud
- Es mayor si el vector es más largo (tiene más valores en más dimensiones)
- Las palabras muy frecuentes (stopwords, por ejemplo) tienen vectores más largos <= co-ocurren con muchas palabras => tienen dot products más altos
- Largo de un vector

$$|\mathbf{v}| = \sqrt{\sum_{i=1}^N v_i^2}$$



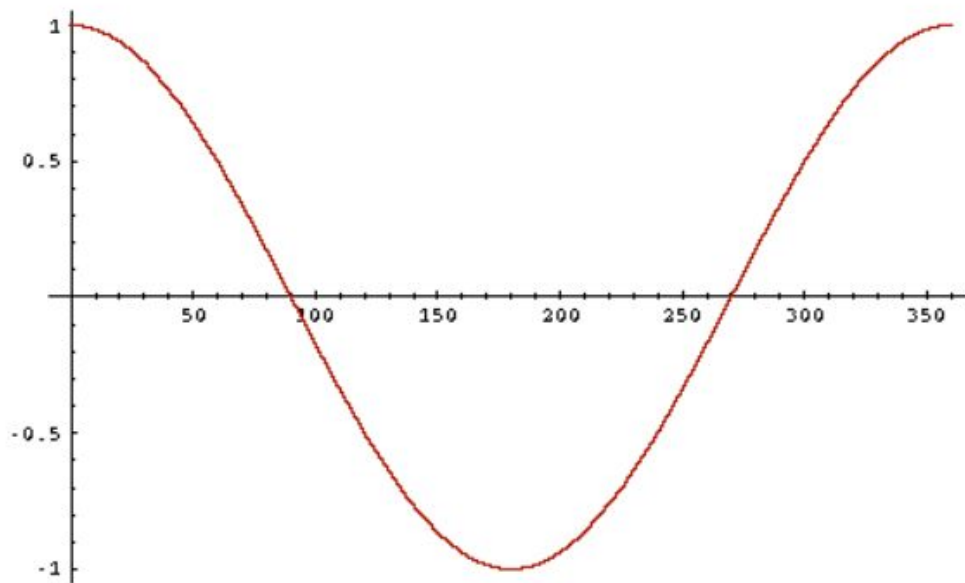
Cálculo de similaridad de palabras: similitud coseno

- **Alternativa:** coseno para normalizar por las longitudes de los vectores

$$\text{cosine}(\vec{v}, \vec{w}) = \frac{\vec{v} \cdot \vec{w}}{|\vec{v}| |\vec{w}|} = \frac{\sum_{i=1}^N v_i w_i}{\sqrt{\sum_{i=1}^N v_i^2} \sqrt{\sum_{i=1}^N w_i^2}}$$

Cálculo de similaridad de palabras: similitud coseno

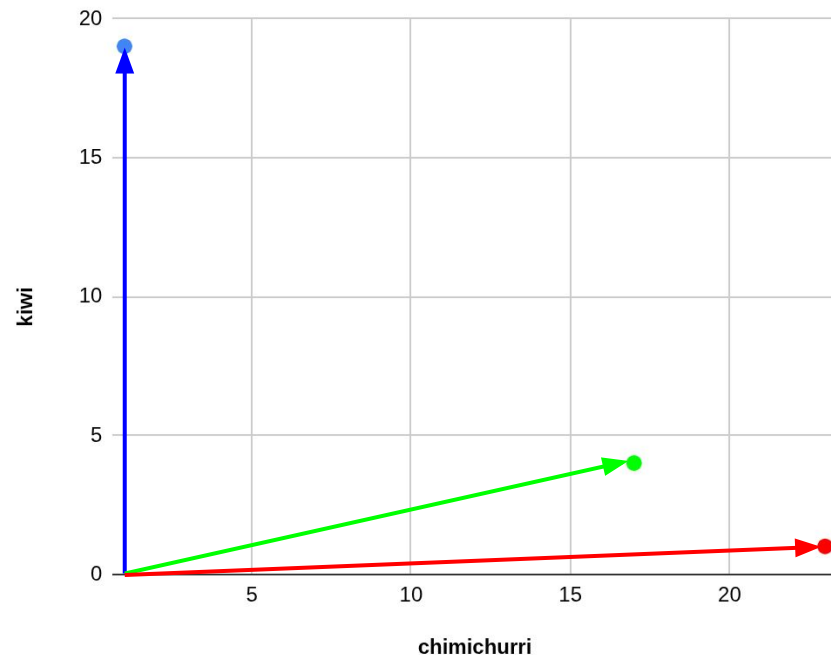
- **+1**: los vectores apuntan a la misma dirección
- **-1**: los vectores apuntan en direcciones opuestas
- **0**: los vectores son ortogonales
- Pero como las frecuencias son $> 0 \Rightarrow$ coseno varía entre 0 y 1.



Cálculo de similaridad de palabras: similitud coseno

- **Ejemplo...** calcularlo a mano...

	chimichurri	kiwi	pera
choripan	23	1	2
vino	17	4	3
uva	1	19	20



Word2vec. Intuición

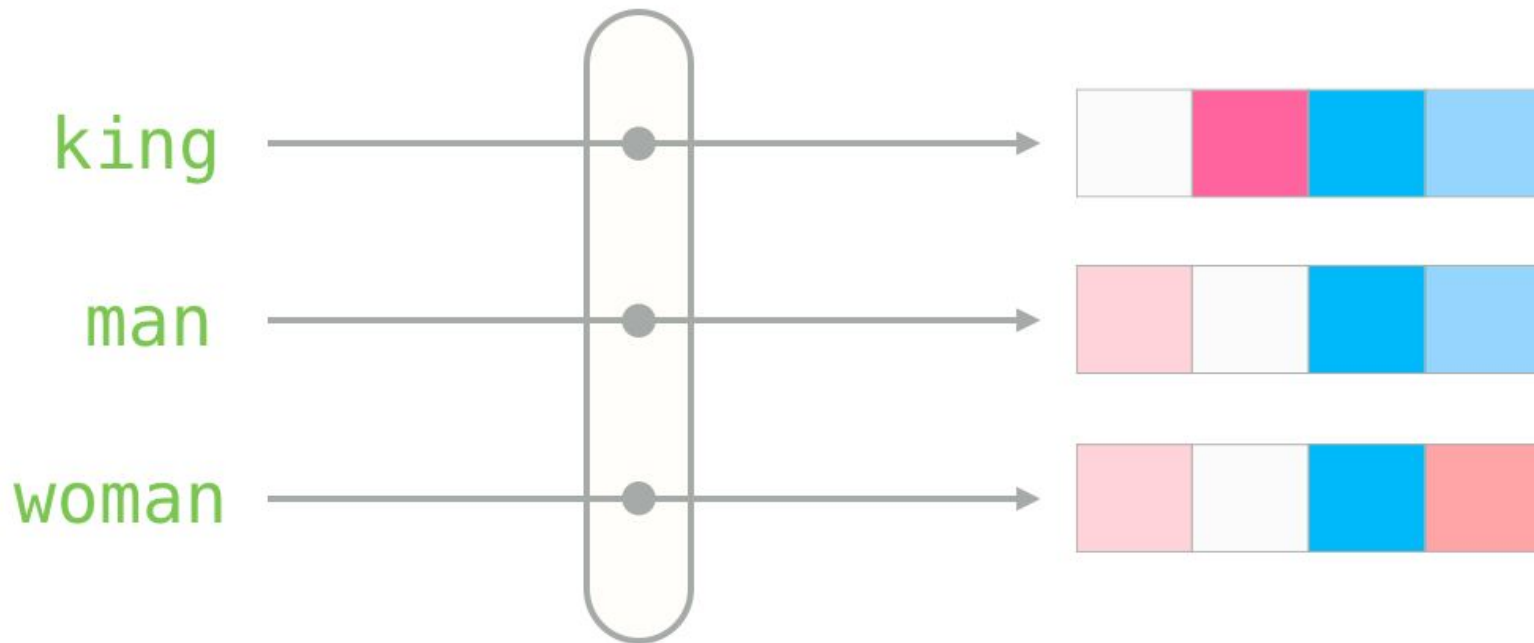


Word embeddings => idea general

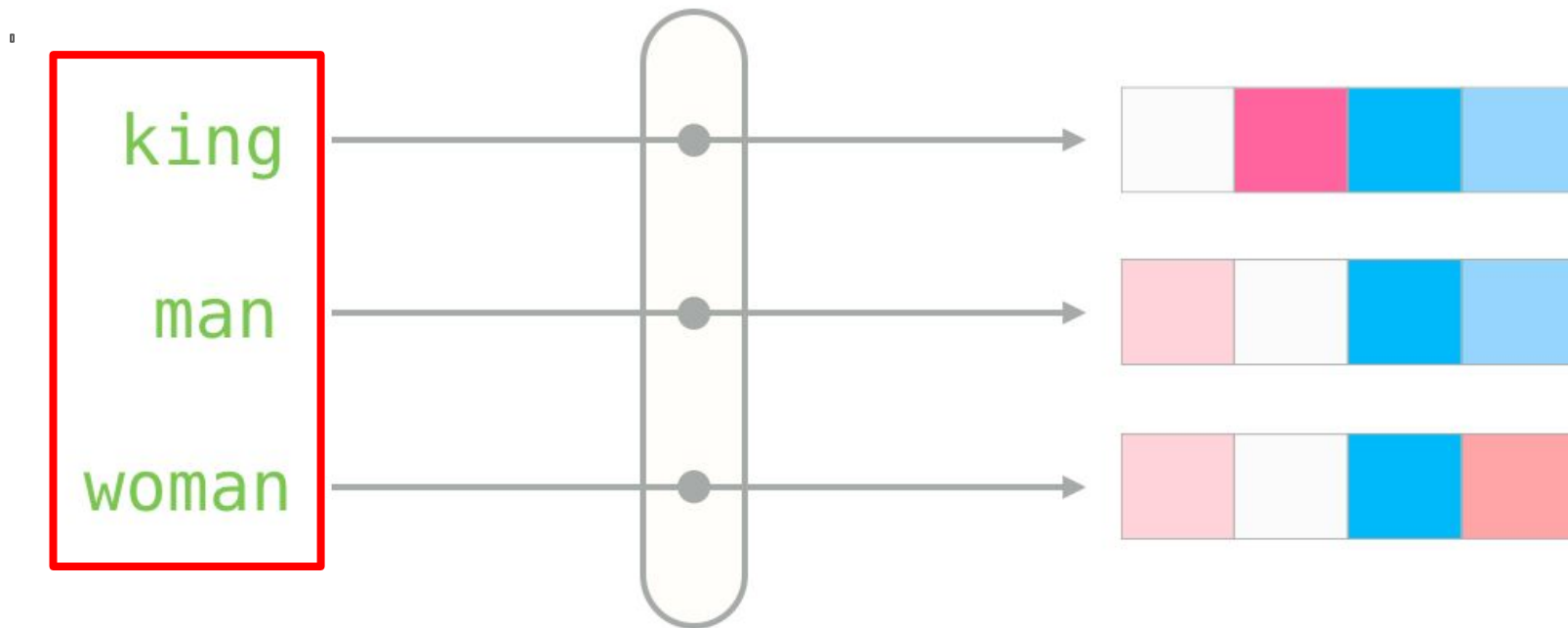
- Reducir la dimensión del vocabulario
 - ~50.000 palabras a ~100 => representación no “esparsa” sino densa
- Flexibilizar supuestos de BoW: cada columna/término/dimensión es un término y se asume independencia
- Hay interacción entre palabras => es esperable que la dimensionalidad sea menor
- Lograr introducir una métrica de distancia para que palabras “cerca” en el nuevo espacio estén “cerca” semánticamente estén cerca.



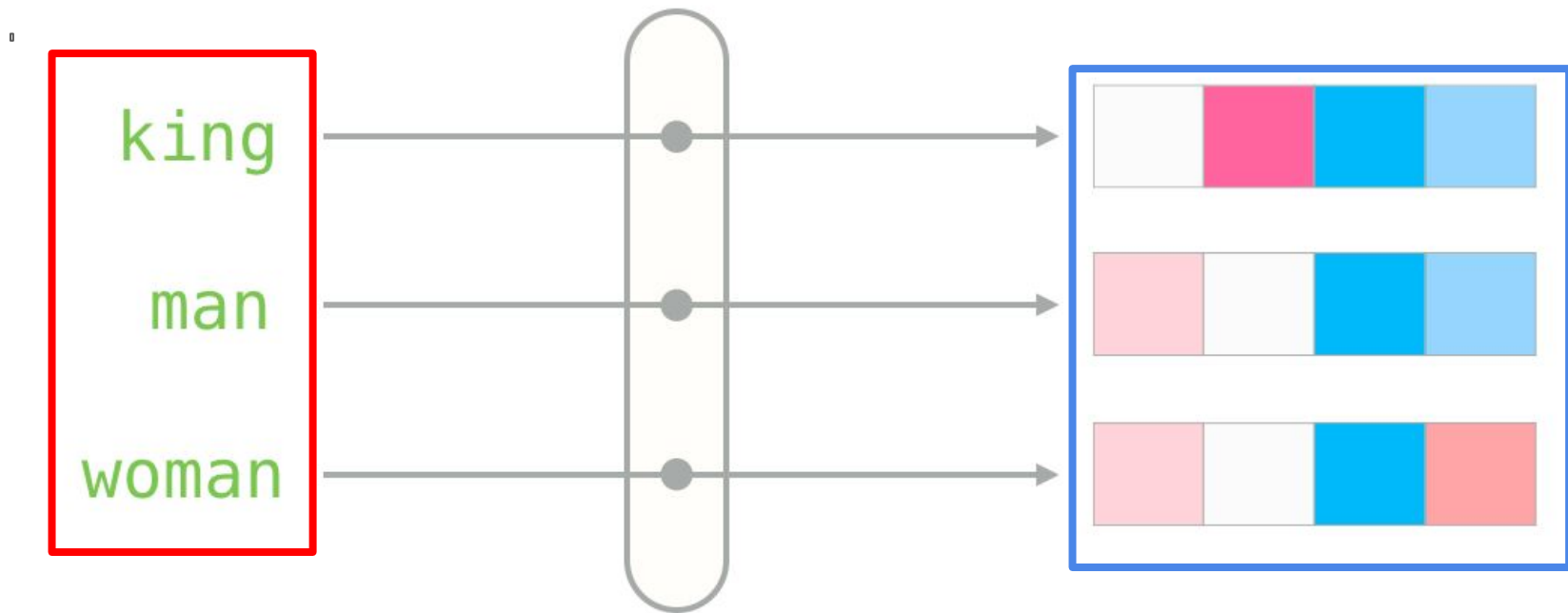
word2vec



word2vec



word2vec



word2vec

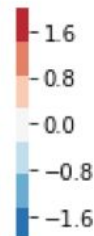
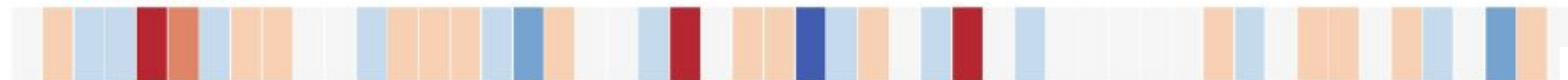
“king”



“Man”



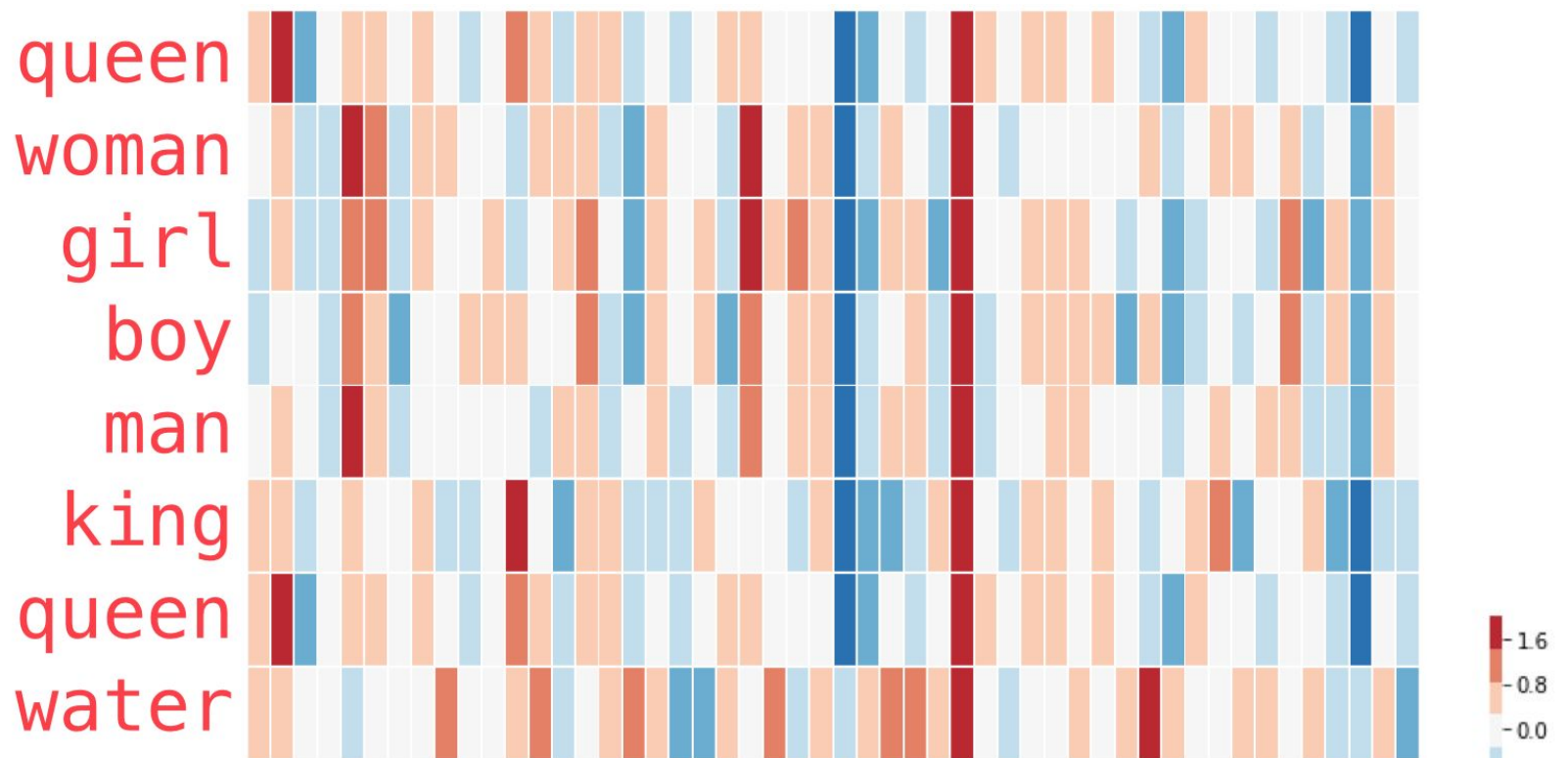
“Woman”



factor-data
EIDAEs_UNSAM

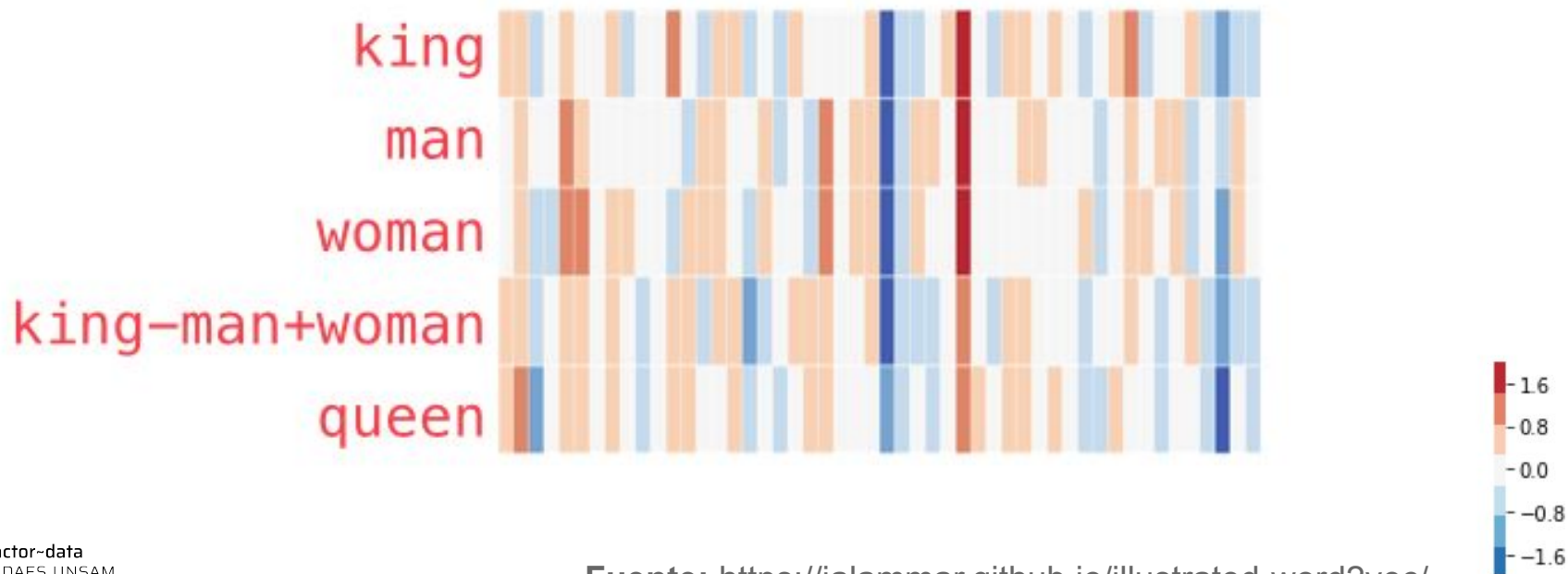
Fuente: <https://jalammar.github.io/illustrated-word2vec/>

word2vec

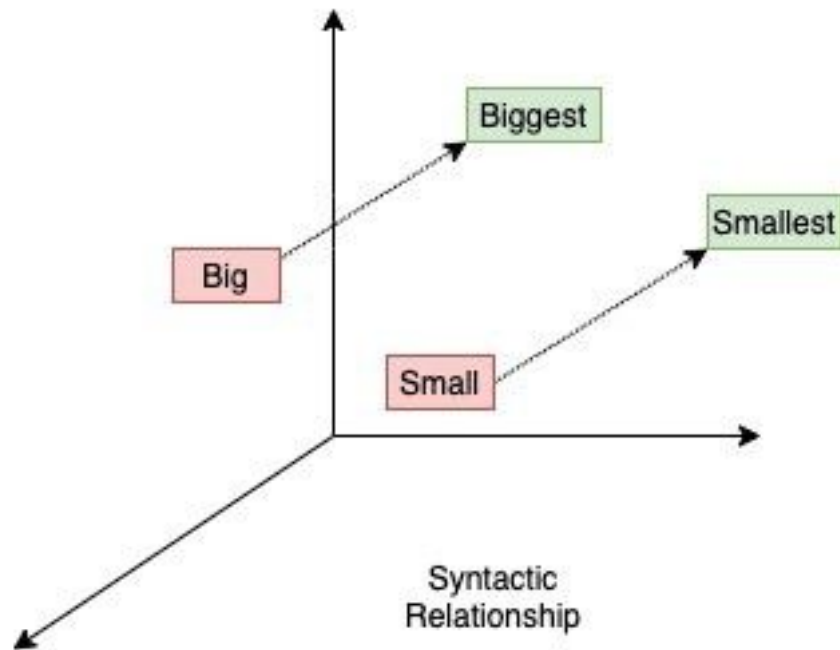
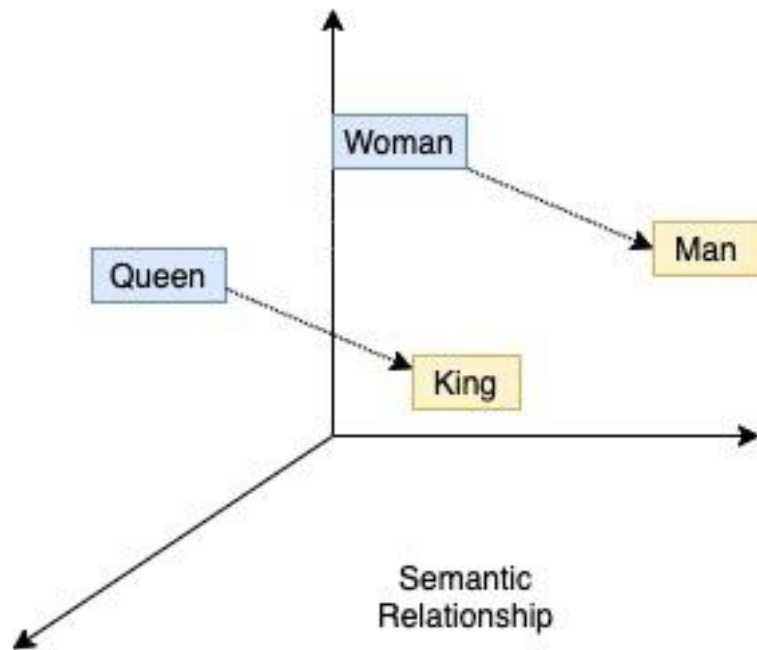


word2vec

king - man + woman $\approx$ queen



word2vec



Evaluación de embeddings

Table 1: *Examples of five types of semantic and nine types of syntactic questions in the Semantic-Syntactic Word Relationship test set.*

Type of relationship	Word Pair 1		Word Pair 2	
Common capital city	Athens	Greece	Oslo	Norway
All capital cities	Astana	Kazakhstan	Harare	Zimbabwe
Currency	Angola	kwanza	Iran	rial
City-in-state	Chicago	Illinois	Stockton	California
Man-Woman	brother	sister	grandson	granddaughter
Adjective to adverb	apparent	apparently	rapid	rapidly
Opposite	possibly	impossibly	ethical	unethical
Comparative	great	greater	tough	tougher
Superlative	easy	easiest	lucky	luckiest
Present Participle	think	thinking	read	reading
Nationality adjective	Switzerland	Swiss	Cambodia	Cambodian
Past tense	walking	walked	swimming	swam
Plural nouns	mouse	mice	dollar	dollars
Plural verbs	work	works	speak	speaks

Evaluación de embeddings

Table 4: Comparison of publicly available word vectors on the Semantic-Syntactic Word Relationship test set, and word vectors from our models. Full vocabularies are used.

Model	Vector Dimensionality	Training words	Accuracy [%]		
			Semantic	Syntactic	Total
Collobert-Weston NNLM	50	660M	9.3	12.3	11.0
Turian NNLM	50	37M	1.4	2.6	2.1
Turian NNLM	200	37M	1.4	2.2	1.8
Mnih NNLM	50	37M	1.8	9.1	5.8
Mnih NNLM	100	37M	3.3	13.2	8.8
Mikolov RNNLM	80	320M	4.9	18.4	12.7
Mikolov RNNLM	640	320M	8.6	36.5	24.6
Huang NNLM	50	990M	13.3	11.6	12.3
Our NNLM	20	6B	12.9	26.4	20.3
Our NNLM	50	6B	27.9	55.8	43.2
Our NNLM	100	6B	34.2	64.5	50.8
CBOW	300	783M	15.5	53.1	36.1
Skip-gram	300	783M	50.0	55.9	53.3

Evaluación de embeddings

Table 8: *Examples of the word pair relationships, using the best word vectors from Table 4 (Skip-gram model trained on 783M words with 300 dimensionality).*

Relationship	Example 1	Example 2	Example 3
France - Paris	Italy: Rome	Japan: Tokyo	Florida: Tallahassee
big - bigger	small: larger	cold: colder	quick: quicker
Miami - Florida	Baltimore: Maryland	Dallas: Texas	Kona: Hawaii
Einstein - scientist	Messi: midfielder	Mozart: violinist	Picasso: painter
Sarkozy - France	Berlusconi: Italy	Merkel: Germany	Koizumi: Japan
copper - Cu	zinc: Zn	gold: Au	uranium: plutonium
Berlusconi - Silvio	Sarkozy: Nicolas	Putin: Medvedev	Obama: Barack
Microsoft - Windows	Google: Android	IBM: Linux	Apple: iPhone
Microsoft - Ballmer	Google: Yahoo	IBM: McNealy	Apple: Jobs
Japan - sushi	Germany: bratwurst	France: tapas	USA: pizza

Usos posibles

- Similitud entre palabras y documentos
- Similitud entre palabras “target” y palabras de contexto al resultado
- Autocompletado
- Traducción automática
- Encontrar clusters de palabras con significados similares
- Buscar analogías entre palabras
- Modelo semántico del lenguaje para comparar con procesamiento del lenguaje hecho por humanos



Aplicaciones en Ciencias Sociales - Estereotipos

The Geometry of Culture: Analyzing the Meanings of Class through Word Embeddings

Austin C. Kozlowski,^a  Matt Taddy,^b
and James A. Evans^{a,c} 

American Sociological Review
2019, Vol. 84(5) 905–949
© American Sociological
Association 2019
DOI: 10.1177/0003122419877135
journals.sagepub.com/home/asr



factor-data
EIDAES_UNSAM

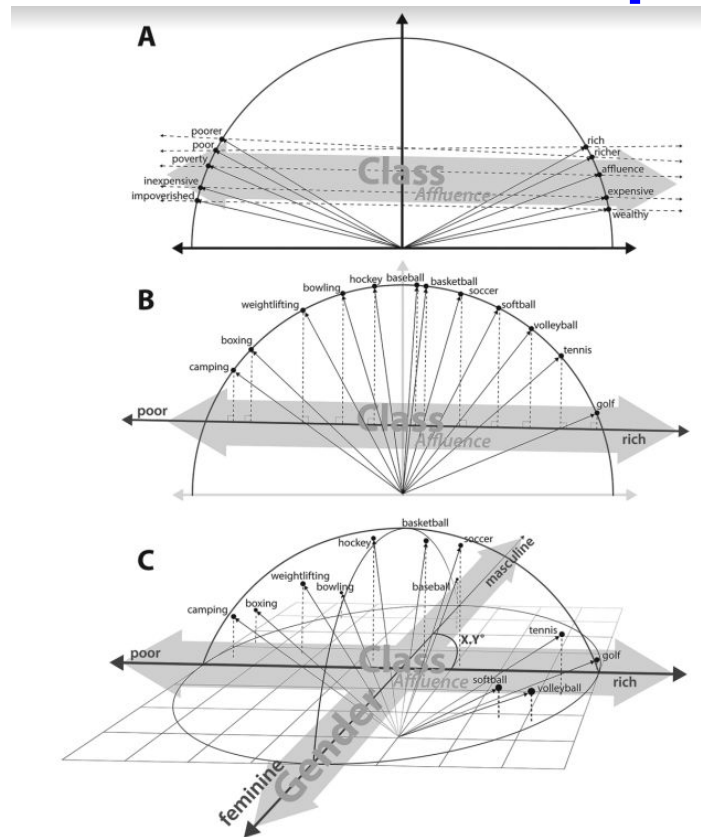
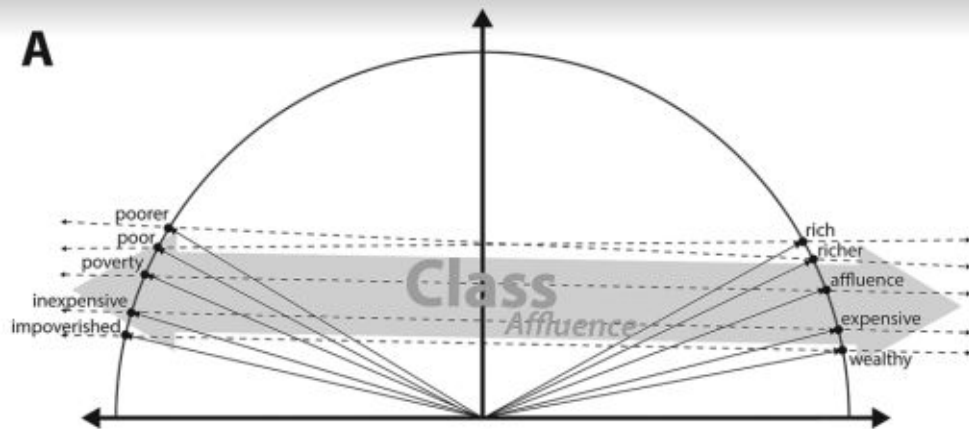


Figure 2. Conceptual Diagram of (A) the Construction of a Cultural Dimension; (B) the Projection of Words onto That Dimension; and (C) the Simultaneous Projection of Words onto Multiple Dimensions

Aplicaciones en Ciencias Sociales - Estereotipos



Measuring Cultural Dimensions

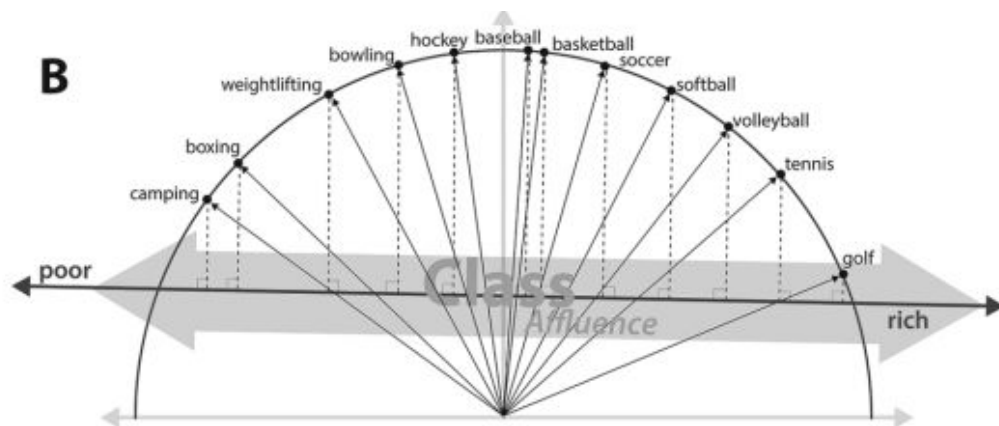
To identify cultural dimensions in word embedding models, we average numerous pairs of antonym words. Cultural dimensions are calculated by simply taking the mean of all word pair differences that approximate a

given dimension, $\frac{\sum_p |\vec{p}_1 - \vec{p}_2|}{|P|}$, where p are

all antonym word pairs in relevant set P , and $\vec{p}_1$ and $\vec{p}_2$ are the first and second word vectors of each pair.¹⁷ The projection of a normalized word vector onto a cultural dimension is calculated with cosine similarity, as is the angle between cultural dimensions.



Aplicaciones en Ciencias Sociales - Estereotipos



Aplicaciones en Ciencias Sociales - Estereotipos

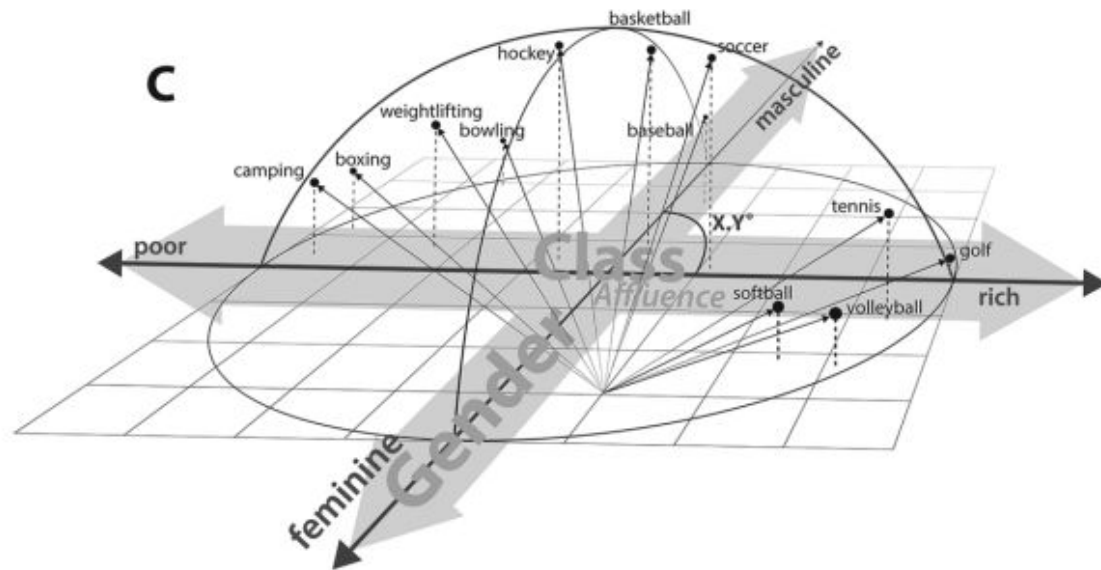


Figure 2. Conceptual Diagram of (A) the Construction of a Cultural Dimension; (B) the Projection of Words onto That Dimension; and (C) the Simultaneous Projection of Words onto Multiple Dimensions



Aplicaciones en Ciencias Sociales - Estereotipos

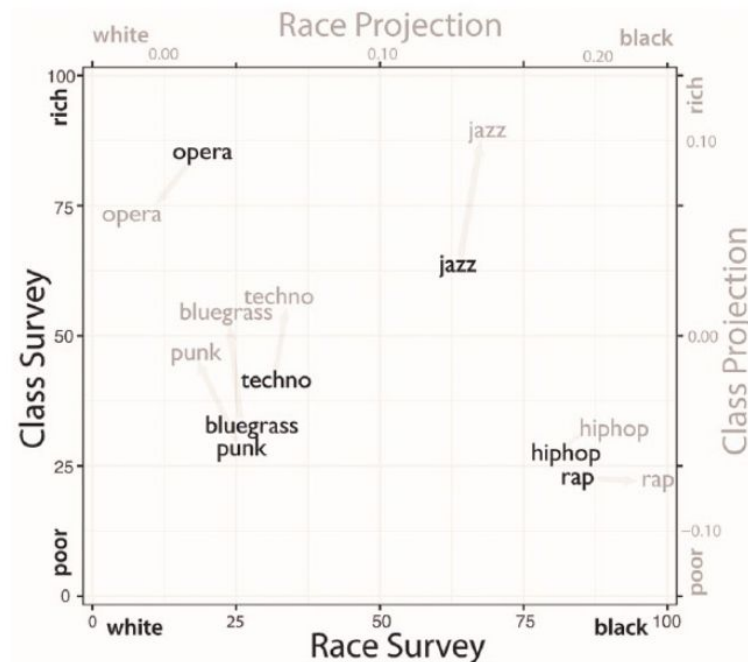


Figure 3. Projection of Music Genres onto Race and Class Dimensions of the Google News Word Embedding (Gray) and Average Survey Ratings for Race and Class Associations (Black)

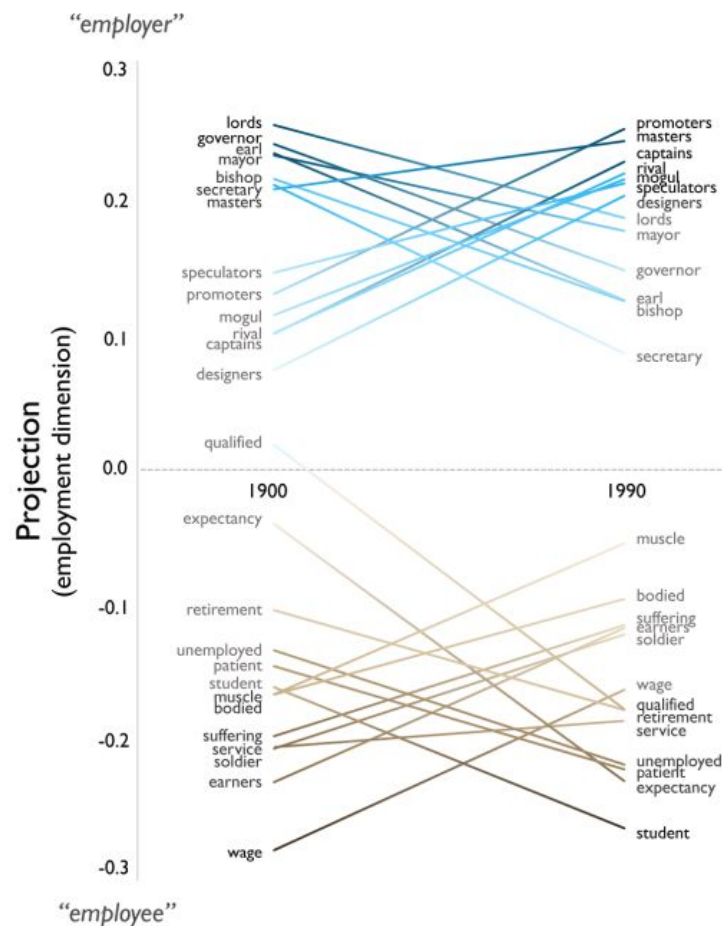


Figure 10. Words That Project High and Low on the Employment Dimension of Word Embedding Models Trained on Texts Published at the Beginning and End of the Twentieth Century; 1900–1919 and 1980–1999 Google Ngrams Corpus

Aplicaciones en Ciencias Sociales

npj Schizophrenia

www.nature.com/npjSch
All rights reserved 2334-265X/15

ARTICLE OPEN

Automated analysis of free speech predicts psychosis onset in high-risk youths

Gillinder Bedi^{1,2,9}, Facundo Carrillo^{3,9}, Guillermo A Cecchi⁴, Diego Fernández Slezak³, Mariano Sigman⁵, Natália B Mota⁶, Sidarta Ribeiro⁶, Daniel C Javitt^{1,7}, Mauro Copelli⁸ and Cheryl M Corcoran^{1,7}

BACKGROUND/OBJECTIVES: Psychiatry lacks the objective clinical tests routinely used in other specializations. Novel computerized methods to characterize complex behaviors such as speech could be used to identify and predict psychiatric illness in individuals.

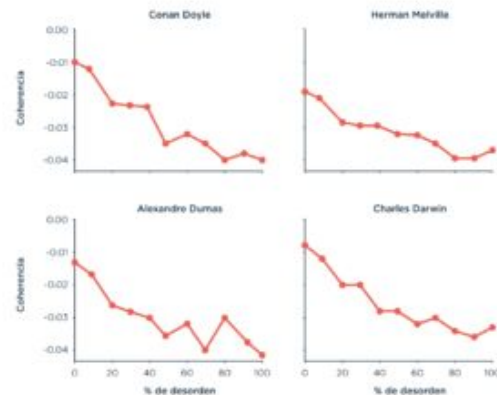
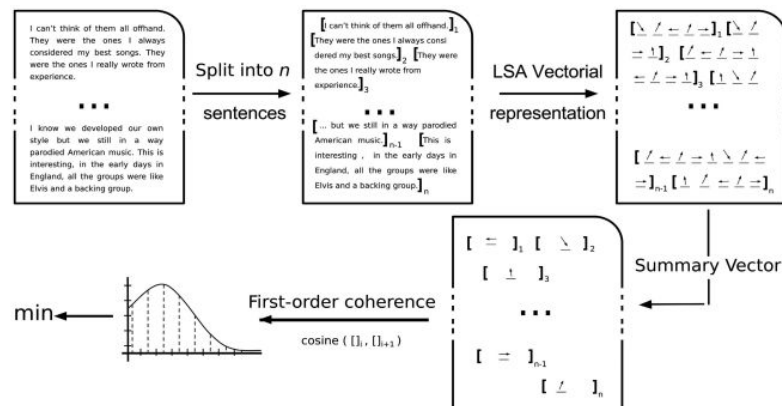
AIMS: In this proof-of-principle study, our aim was to test automated speech analyses combined with Machine Learning to predict later psychosis onset in youths at clinical high-risk (CHR) for psychosis.

METHODS: Thirty-four CHR youths (11 females) had baseline interviews and were assessed quarterly for up to 2.5 years; five transitioned to psychosis. Using automated analysis, transcripts of interviews were evaluated for semantic and syntactic features predicting later psychosis onset. Speech features were fed into a convex hull classification algorithm with leave-one-subject-out cross-validation to assess their predictive value for psychosis outcome. The canonical correlation between the speech features and prodromal symptom ratings was computed.

RESULTS: Derived speech features included a Latent Semantic Analysis measure of semantic coherence and two syntactic markers of speech complexity: maximum phrase length and use of determiners (e.g., *which*). These speech features predicted later psychosis development with 100% accuracy, outperforming classification from clinical interviews. Speech features were significantly correlated with prodromal symptoms.

CONCLUSIONS: Findings support the utility of automated speech analysis to measure subtle, clinically relevant mental state changes in emergent psychosis. Recent developments in computer science, including natural language processing, could provide the foundation for future development of objective clinical tests for psychiatry.

npj Schizophrenia (2015) 1, Article number: 15030; doi:10.1038/npjSch.2015.30; published online 26 August 2015



Aplicaciones en Ciencias Sociales - Estereotipos

Semantics derived automatically
from language corpora contain
human-like biases

Aylin Caliskan,^{1*} Joanna J. Bryson,^{1,2*} Arvind Narayanan^{1*}

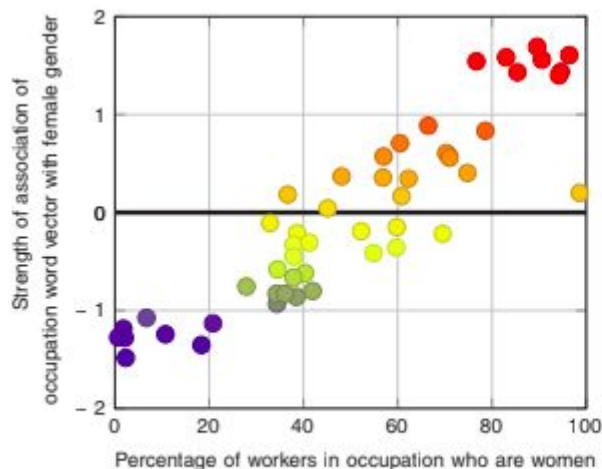


Fig. 1. Occupation-gender association. Pearson's correlation coefficient $\rho = 0.90$ with $P < 10^{-18}$.

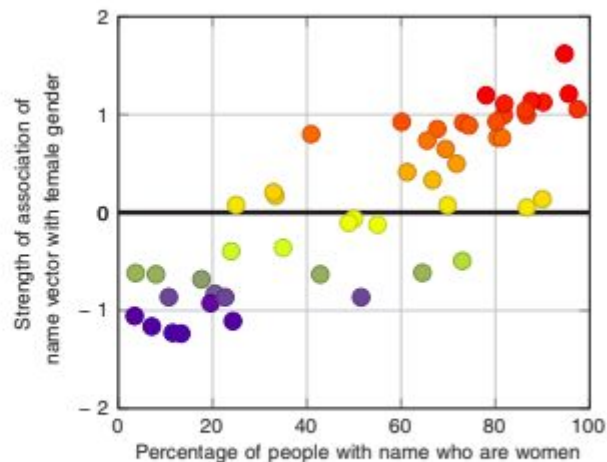


Fig. 2. Name-gender association. Pearson's correlation coefficient $\rho = 0.84$ with $P < 10^{-13}$.



Aplicaciones en Ciencias Sociales - Trayectorias

- Arquitectura de embeddings / transformers como forma de generar representaciones comprimidas de trayectorias en múltiples dimensiones.
- Life2Vec
 - Dataset n~3.000.000 de habitantes
 - Historias laborales / ingresos
 - Historias migratorias
 - Historias de salud
- Tarea: predicción de fallecimiento
- Usan algo similar a lo que funciona por detrás de chatGPT

nature computational science

Article

<https://doi.org/10.1038/s43588-023-00435-1>

Using sequences of life-events to predict human lives

Received: 6 June 2023

Accepted: 15 November 2023

Published online: 18 December 2023

 Check for updates

Germans Savcicens¹, Tina Eliassi-Rad^{2,3}, Lars Kai Hansen¹,
Laust Hvas Mortensen^{4,5}, Lau Lilleholt^{6,7}, Anna Rogers⁸, Ingo Zettler^{6,7} &
Sune Lehmann^{1,2} 

Here we represent human lives in a way that shares structural similarity to language, and we exploit this similarity to adapt natural language processing techniques to examine the evolution and predictability of human lives based on detailed event sequences. We do this by drawing on a comprehensive registry dataset, which is available for Denmark across several years, and that includes information about life-events related to health, education, occupation, income, address and working hours, recorded with day-to-day resolution. We create embeddings of life-events in a single vector space, showing that this embedding space is robust and highly structured. Our models allow us to predict diverse outcomes ranging from early mortality to personality nuances, outperforming state-of-the-art models by a wide margin. Using methods for interpreting deep learning models, we probe the algorithm to understand the factors that enable our predictions. Our framework allows researchers to discover potential mechanisms that impact life outcomes as well as the associated possibilities for personalized interventions.

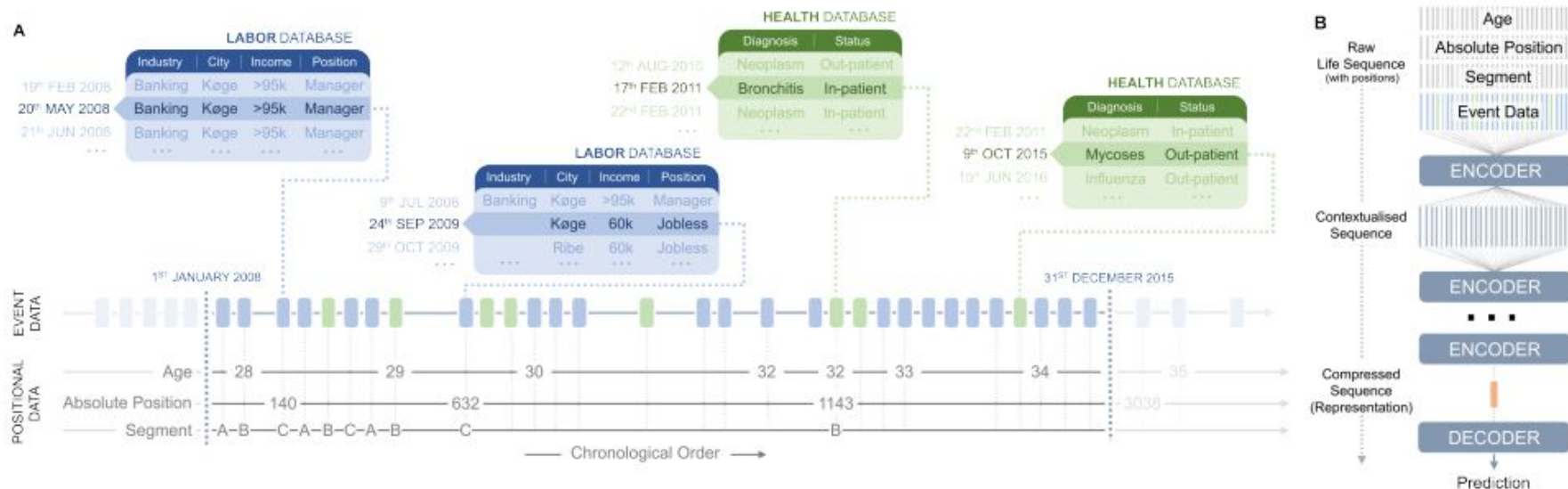
We live in the age of algorithm-driven prediction of human behavior. The predictions range from those at the global and population level, with societies allocating vast resources to predicting phenomena such as global warming¹ or the spread of infectious diseases², all the way to the constant flow of individual micro-predictions that shape our reality and behavior as we use social media³. When it comes to individual life outcomes, however, the picture is more complex. Sociodemographic

decade interval, we show that accurate individual predictions are indeed possible. Our dataset includes a host of indicators, such as health, professional occupation and affiliation, income level, residency, working hours and education (Dataset section).

The main reason why we are currently experiencing this 'age of human prediction' is the advent of massive datasets and powerful machine learning algorithms^{4,5}. Over the past decade, machine learning



Aplicaciones en Ciencias Sociales - Trayectorias



[CLS] [MALE] [YEAR=1957]
[SEP] ... [MANAGER] [SEP] ...
[FRACTURE] [IN-PATIENT]
[SEP]

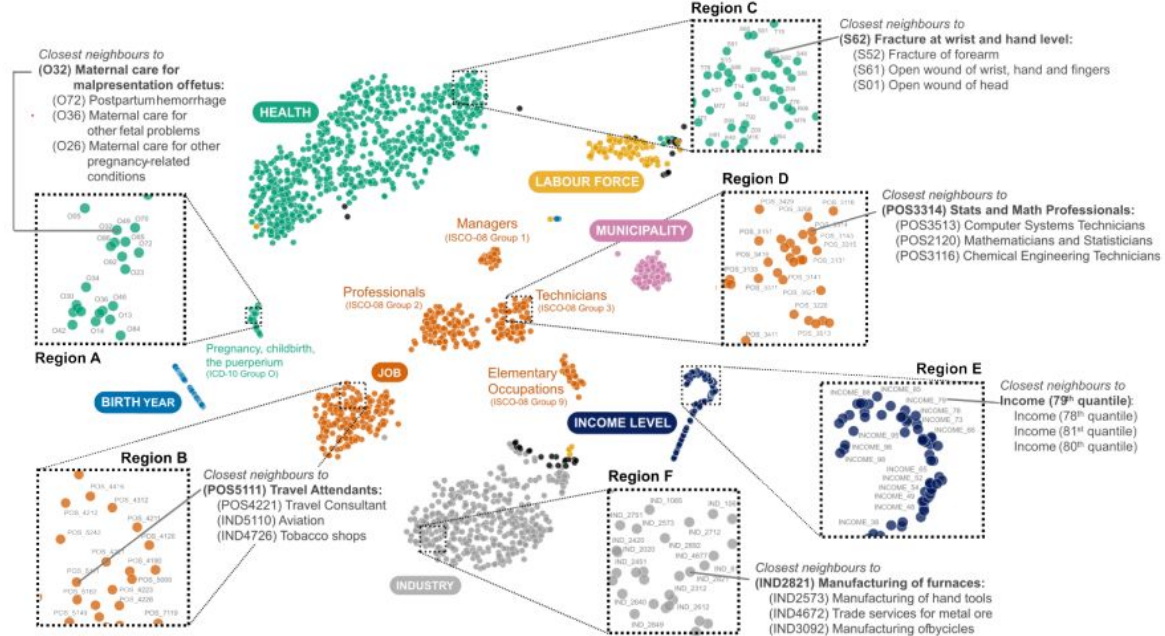


Figure 4: Two-dimensional projection of the concept space (using the PaCMAP [72]). Each point corresponds to a concept token in the vocabulary. Points are colored based on the concept types (several types are omitted - black points). Each region provides a closer look at several parts of the concept space. You can also see the top three closest neighbors for selected tokens (based on the cosine distance). (A) Diagnoses related to Pregnancy, childbirth, and the puerperium in ICD-10 [40]. (B) Job concepts related to Service and Sales Workers (corresponds to Job Category 5 of ISCO-08 [38]). (C) Injury-related diagnoses in ICD-10 [40]. (D) Job concepts related to Technicians and Associate Professionals (corresponds to Job Category 3 of ISCO-08 [38]). (E) Income-related concepts. *life2vec* arranges these concepts in increasing ordinal order. (F) Concepts related to the manufacturing industry in DB07 [39].

Aplicaciones en otras disciplinas

arXiv:2507.22291v1 [cs.CV] 29 Jul 2025

Google DeepMind

2025-7-31

AlphaEarth Foundations: An embedding field model for accurate and efficient global mapping from sparse label data

Christopher F. Brown^{1,3}, Michal R. Kazmierski^{1,3}, Valerie J. Pasquella^{1,3}, William J. Rucklidge², Masha Samikova¹, Chenhui Zhang¹, Ivan Shelhamer¹, Estefania Lahera², Olivia Wiles¹, Simon Ilyushchenko², Noel Gorelick¹, Lihui Lydia Zhang¹, Sophia APJ, Emily Schechter², Sean Askay², Oliver Guinan², Rebecca Moore², Alexis Boukouvalas¹ and Pushmeet Kohli¹
¹Equal contributions, ²Google DeepMind, ³Google

Unprecedented volumes of Earth observation data are continually collected around the world, but high-quality labels remain scarce given the effort required to make physical measurements and observations. This has led to considerable investment in bespoke modeling efforts translating sparse labels into maps. Here we introduce AlphaEarth Foundations, an embedding field model yielding a highly general, geospatial representation that assimilates spatial, temporal, and measurement contexts across multiple sources, enabling accurate and efficient production of maps and monitoring systems from local to global scales. The embeddings generated by AlphaEarth Foundations are the only to consistently outperform all previous featurization approaches tested on a diverse set of mapping evaluations without re-training. We will release a dataset of global, annual, analysis-ready embedding field layers from 2017 through 2024.

Introduction

Management of global food supplies, public health, and disaster response all start from maps that geographically anchor questions like “which forests pose an unacceptable wildfire risk?” or “where are soybeans grown?”. The launch of the first Landsat satellite in 1972 marked the dawn of an era where spaceborne monitoring could serve the interests of global environmental policy-making and provide critical insights into our changing planet (Cohen and Goward, 2004). Over the following decades Earth observation (EO) data became widely available, and streams from both historic and modern EO instruments are now routinely used to create maps that answer questions about the past, present, and future of Earth’s ecosystems and climate (Gorelick et al., 2020).

embedding model that solves fundamental challenges in the institution of mapping through the generation of a universal feature space. The features produced by our model consistently achieve top performance in all application domains tested when compared to other general and even domain specific approaches (Figure 1A). This marks a shift from the previous state-of-the-art for which no single approach was dominant.

From sparse labels to maps

High-quality maps depend on high-quality labeled data, yet when working at global scales, a balance must be struck between measurement precision and spatial coverage. Many global mapping efforts focus on individual ecosystems like forests (Hansen et al., 2013), water (Pokel et al.,

Satellite Embedding V1



Dataset Availability

2017-01-01T00:00:00Z~2024-01-01T00:00:00Z

Dataset Provider

[Google Earth Engine](#) [Google DeepMind](#)

Earth Engine Snippet

```
ee.ImageCollection("GOOGLE/SATELLITE_EMBEDDING/V1/ANNUAL")
```

Description

Bands

Image Properties

Terms of Use

The Google Satellite Embedding dataset is a global, analysis-ready collection of learned geospatial [embeddings](#). Each 10-meter pixel in this dataset is a 64-dimensional representation, or “[embedding vector](#),” that encodes temporal trajectories of surface conditions at and around that pixel as measured by various Earth observation instruments and datasets, over a single calendar year. Unlike conventional spectral inputs and indices, where bands reflect physical measurements, embeddings are feature vectors that summarize relationships across multi-source, multi-modal observations in a less directly interpretable, but more powerful way.



factor-data
EIDAES_UNSAM

Aplicaciones en otras disciplinas

Google DeepMind

2025-7-31

AlphaEarth Foundations: An embedding field model for accurate and efficient global mapping from sparse label data

Christopher F. Brown^{1,3}, Michal R. Kazmierski^{1,3}, Valerie J. Pasquarella^{2,3}, William J. Rucklidge², Masha Samikova¹, Chenzhi Zhang¹, Ivan Shekhar¹, Estefania Lahera¹, Olivia Wiles¹, Simon Hyushchenko², Noel Gorelick¹, Lihui Lydia Zhang¹, Sophia APJ, Emily Schechter², Sean Askay², Oliver Guinan², Rebecca Moore², Alexis Boukouvalas¹ and Pushmeet Kohli¹

¹Equal contributions, ²Google DeepMind, ³Google

Unprecedented volumes of Earth observation data are continually collected around the world, but high-quality labels remain scarce given the effort required to make physical measurements and observations. This has led to considerable investment in bespoke modeling efforts translating sparse labels into maps. Here we introduce AlphaEarth Foundations, an embedding field model yielding a highly general, geospatial representation that assimilates spatial, temporal, and measurement contexts across multiple sources, enabling accurate and efficient production of maps and monitoring systems from local to global scales. The embeddings generated by AlphaEarth Foundations are the only to consistently outperform all previous featurization approaches tested on a diverse set of mapping evaluations without re-training. We will release a dataset of global, annual, analysis-ready embedding field layers from 2017 through 2024.

Introduction

Management of global food supplies, public health, and disaster response all start from maps that geographically anchor questions like “which forests pose an unacceptable wildfire risk?” or “where are soybeans grown?”. The launch of the first Landsat satellite in 1972 marked the dawn of an era where spaceborne monitoring could serve the interests of global environmental policy-making and provide critical insights into our changing planet (Cohen and Goward, 2004). Over the following decades Earth observation (EO) data became widely available, and streams from both historic and modern EO instruments are now routinely used to create maps that answer questions about the past, present, and future of Earth’s ecosystems and climate (Gorelick et al., 2020).

embedding model that solves fundamental challenges in the institution of mapping through the generation of a universal feature space. The features produced by our model consistently achieve top performance in all application domains tested when compared to other general and even domain specific approaches (Figure 1A). This marks a shift from the previous state-of-the-art for which no single approach was dominant.

From sparse labels to maps

High-quality maps depend on high-quality labeled data, yet when working at global scales, a balance must be struck between measurement precision and spatial coverage. Many global mapping efforts focus on individual ecosystems like forests (Hansen et al., 2013), water (Pokel et al.,

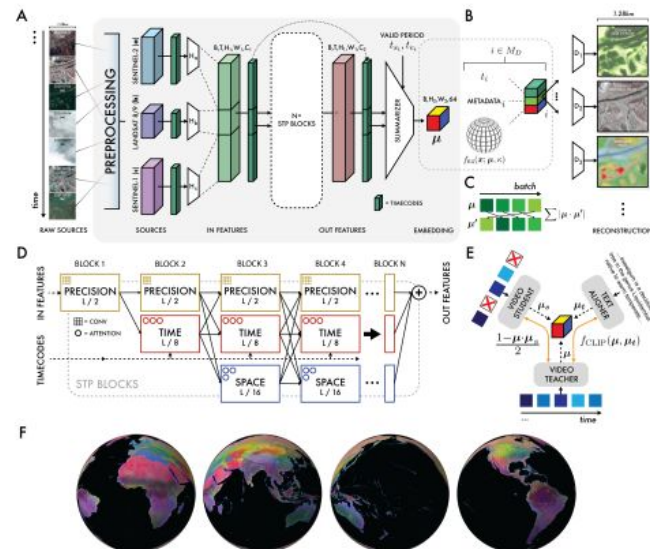


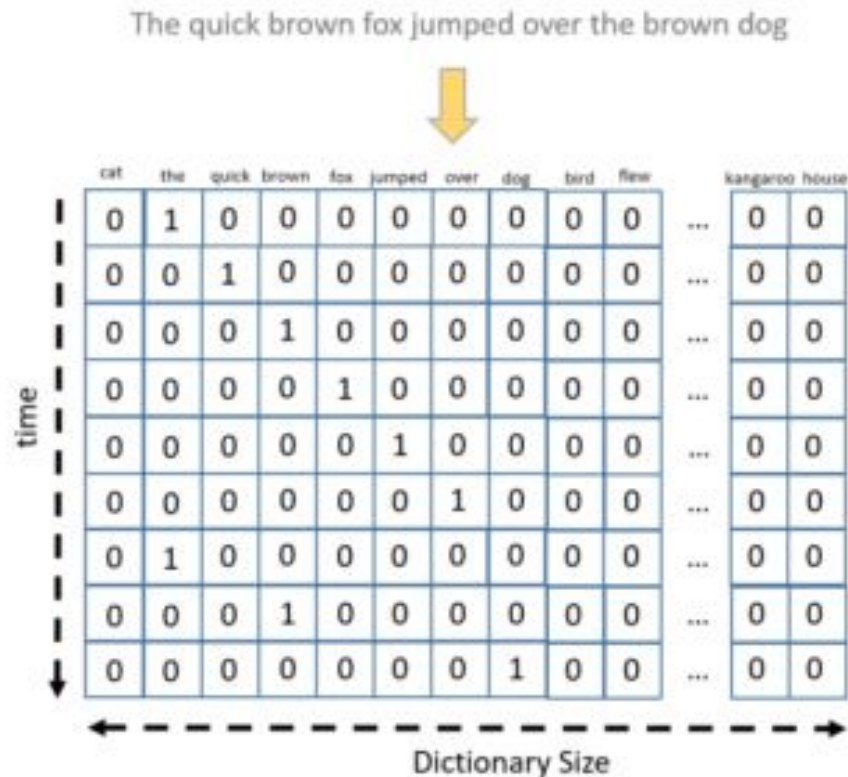
Figure 2 | AlphaEarth Foundations. (A) Block diagram of the overall network architecture used for video analysis. Preprocessing converts raw observation data via normalization using global statistics, and acquisition timestamps are converted to sinusoidal timecodes. Individual source encoders transform inputs to the same latent space before entering the bulk of the model. Outputs are summarized using conditional timecodes or “summary periods”, unique to each decoded source and contrastive learning task. μ refers to the embedding outputs of the model. (B) Model outputs are treated as the mean direction of a von Mises-Fisher distribution, and decoding proceeds by sampling this distribution, and concatenating it with sensor geometry metadata and a timecode indicating the relative position in the valid period to decode. Decoding proceeds for all sources, with losses dependent on the characteristics of each source (see supplemental materials S1). (C) To prevent collapse and improve performance, embeddings are compared to equivalent batch-rotated embeddings using a dot product. The absolute value of this quantity is minimized as a necessary condition for an empirically uniform distribution in S^{63} . (D) Block diagram of the model bulk, consisting of simultaneous pathways at different resolutions to maintain efficiency and spatial precision. (E) Contrastive learning between the video teacher and student model, and text encoder. (F) Complete 360° view of 2023 annual embedding field covering Earth’s land surface including minor islands over approximately $\pm 82^\circ$.

¿Cómo sucede la magia?



One hot encoding

- Eje Y = tiempo
- Eje X = vocabulario
- Celdas: 1 si la palabra aparece en ese “momento”; 0 si no aparece



Skip-gram

Cambia la unidad

Ahora el corpus es visto como un todo continuo...

No se ven los documentos por separado

Un parámetro importante: el tamaño de la ventana...

Otro metodo: CBOW (al revés)

Source Text

Training Samples

The quick brown fox jumps over the lazy dog. →

(the, quick)
(the, brown)

The quick brown fox jumps over the lazy dog. →

(quick, the)
(quick, brown)
(quick, fox)

The quick brown fox jumps over the lazy dog. →

(brown, the)
(brown, quick)
(brown, fox)
(brown, jumps)

The quick brown fox jumps over the lazy dog. →

(fox, quick)
(fox, brown)
(fox, jumps)
(fox, over)



Skip-gram

Contexte				Mot Cible
The	Quick	Fox	Jump	Brown
Quick	Brown	Jumps	Over	Fox
Brown	Fox	Over	The	Jumps



Skip-gram - Matriz de co-ocurrencias

	brown	dog	fox	jumps	lazy	over	quick	the
brown	0	0	0	0	0	0	1	1
dog	0	0	0	0	1	0	0	1
fox	1	0	0	0	0	0	1	0
jumps	1	0	1	0	0	0	0	0
lazy	0	0	0	0	0	1	0	1
over	0	0	1	1	0	0	0	0
quick	0	0	0	0	0	0	0	1
the	0	0	0	1	0	1	0	0

Skip-gram (otro ejemplo)

Thou shalt not make a machine in the likeness of a human mind

thou	shalt	not	make	a	machine	in	the	...
------	-------	-----	------	---	---------	----	-----	-----

Cambia la unidad

Ahora el corpus es visto como
un todo continuo...

No se ven los documentos por
separado

Un parámetro importante: el
tamaño de la ventana...

Otro metodo: CBOW (al revés)

input word	target word
not	thou
not	shalt
not	make
not	a



Skip-gram (otro ejemplo)

Thou shalt not make a machine in the likeness of a human mind

thou	shalt	not	make	a	machine	in	the	...
------	-------	-----	------	---	---------	----	-----	-----

thou	shalt	not	make	a	machine	in	the	...
------	-------	-----	------	---	---------	----	-----	-----

input word	target word
not	thou
not	shalt
not	make
not	a
make	shalt
make	not
make	a
make	machine

Skip-gram (otro ejemplo)

Thou shalt not make a machine in the likeness of a human mind

thou	shalt	not	make	a	machine	in	the	...
------	-------	-----	------	---	---------	----	-----	-----

thou	shalt	not	make	a	machine	in	the	...
------	-------	-----	------	---	---------	----	-----	-----

thou	shalt	not	make	a	machine	in	the	...
------	-------	-----	------	---	---------	----	-----	-----

thou	shalt	not	make	a	machine	in	the	...
------	-------	-----	------	---	---------	----	-----	-----

thou	shalt	not	make	a	machine	in	the	...
------	-------	-----	------	---	---------	----	-----	-----

input word	target word
not	thou
not	shalt
not	make
not	a
make	shalt
make	not
make	a
make	machine
a	not
a	make
a	machine
a	in
machine	make
machine	a
machine	in
machine	the
in	a
in	machine
in	the
in	likeness



Modelando con skipgram

input word	target word
not	thou
not	shalt
not	make
not	a
make	shalt
make	not
make	a
make	machine
a	not
a	make
a	machine
a	in
machine	make
machine	a
machine	in
machine	the
in	a
in	machine
in	the
in	likeness

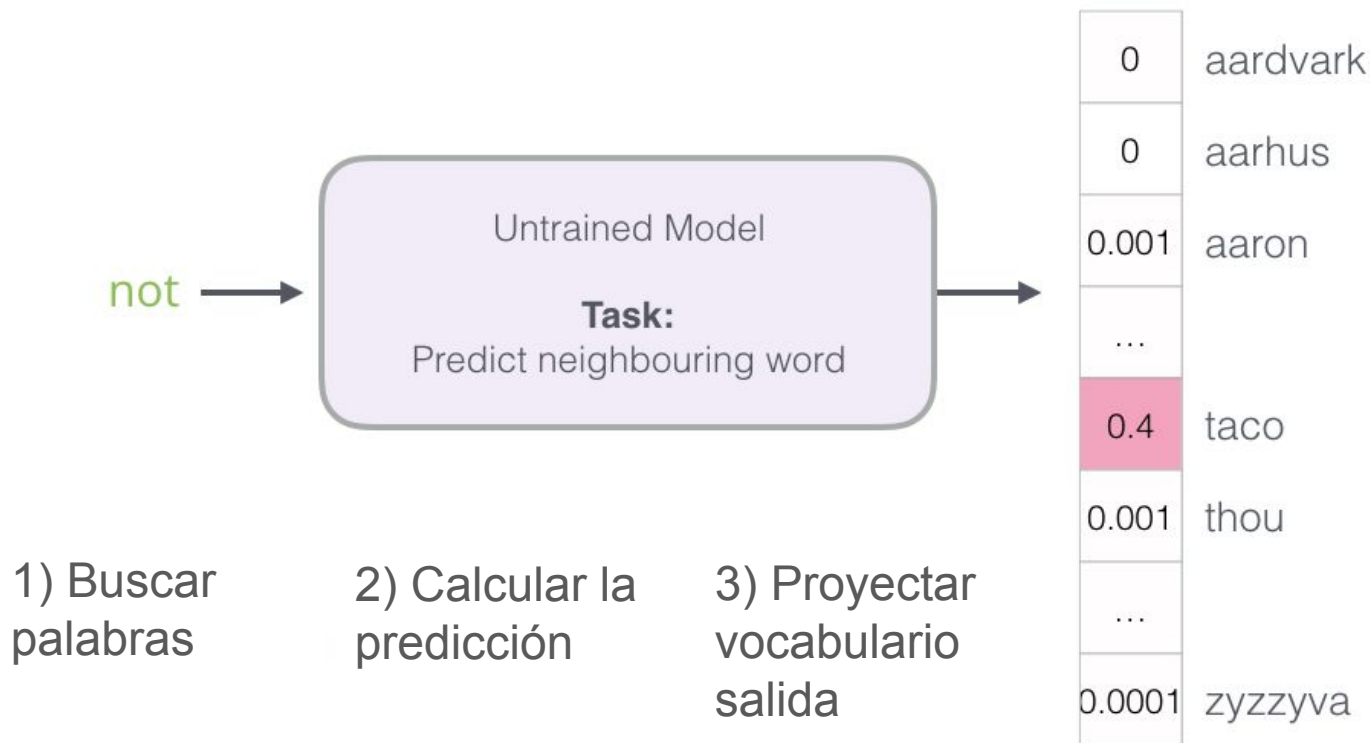
not →

Untrained Model

Task:
Predict neighbouring word



Modelando con skipgram



Modelando con skipgram

Actual
Target

0
0
0
...
0
1
...
0

-

Model
Prediction

0	aardvark
0	aarhus
0.001	aaron
...	
0.4	taco
0.001	thou
...	
0.0001	zyzzyva



Modelando con skipgram

Actual
Target

0
0
0
...
0
1
...
0

-

Model
Prediction

0	aardvark
0	aarhus
0.001	aaron
...	...
0.4	taco
0.001	thou
...	...
0.0001	zyzzyva

=

Error

0
0
-0.001
...
-0.4
0.999
...
-0.0001

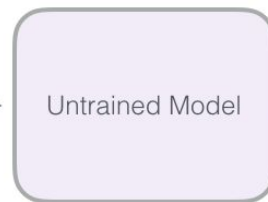


Modelando con skipgram

Actual
Target

0
0
0
...
0
1
...
0

not



Model
Prediction

0	aardvark
0	aarhus
0.001	aaron
...	...
0.4	taco
0.001	thou
...	...
0.0001	zyzzyva

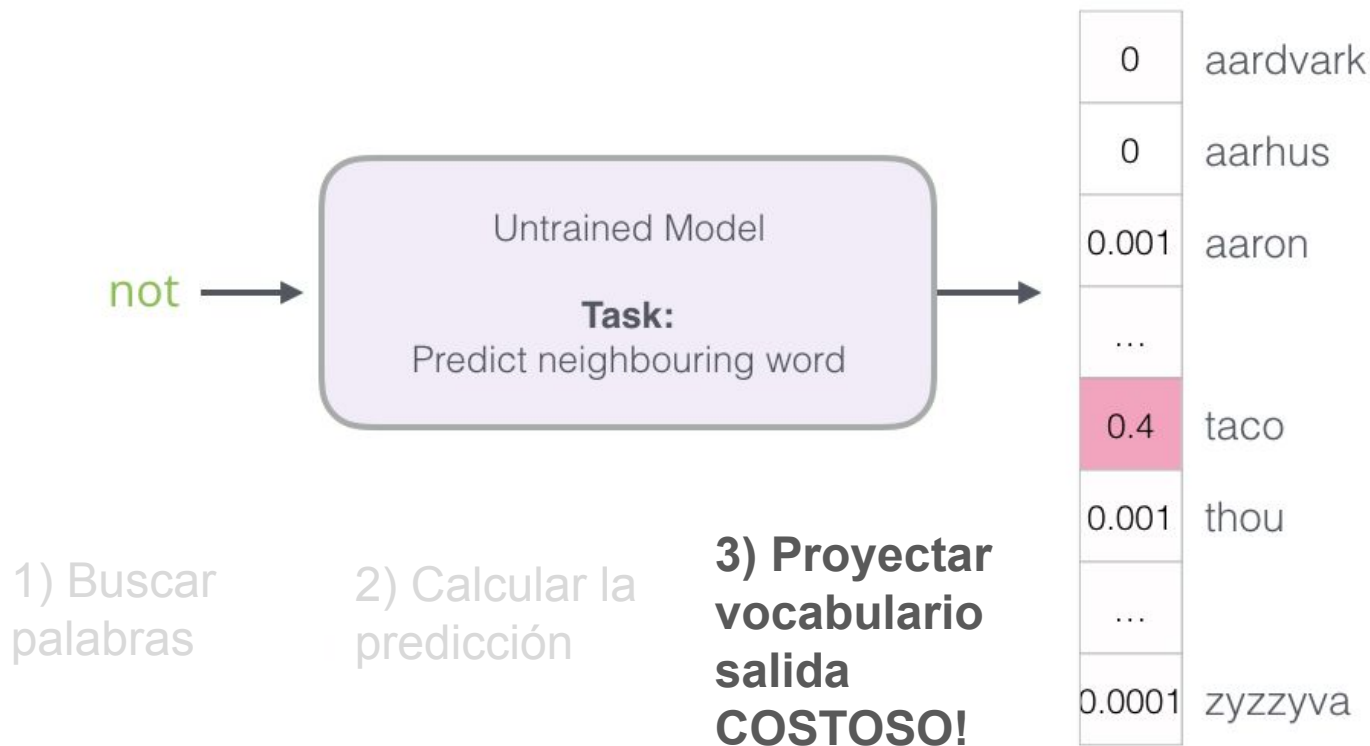
Error

0
0
-0.001
...
-0.4
0.999
...
-0.0001

Update
Model
Parameters

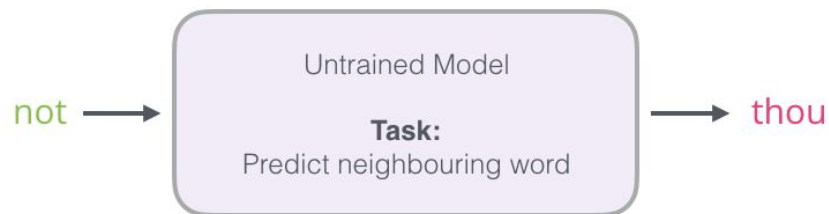


Modelando con skipgram => PROBLEMA



Modelando con skipgram => PROBLEMA

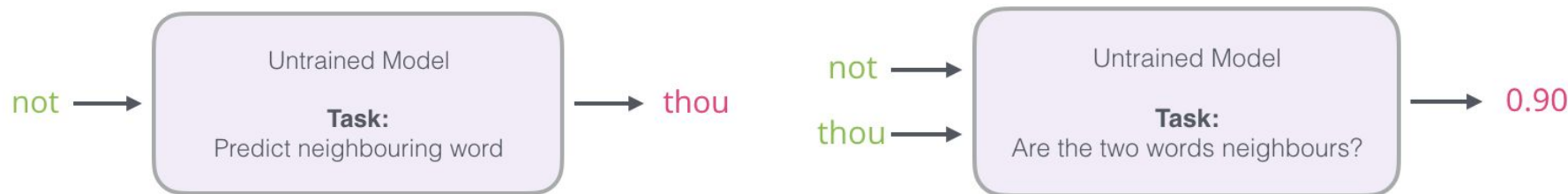
Change Task from



Modelando con skipgram => PROBLEMA

To:

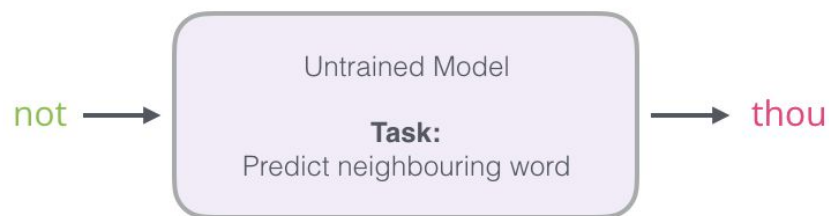
Change Task from



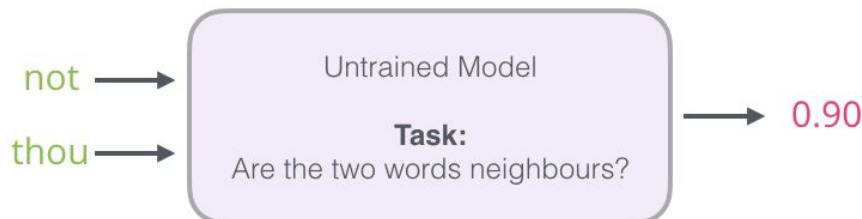
Modelando con skipgram => PROBLEMA

To:

Change Task from



input word	target word
not	thou
not	shalt
not	make
not	a
make	shalt
make	not
make	a
make	machine



input word	output word	target
not	thou	1
not	shalt	1
not	make	1
not	a	1
make	shalt	1
make	not	1
make	a	1
make	machine	1

Problema!
Todos
ejemplos
positivos...

OVERFITTING

Negative sampling

input word	output word	target
not	thou	1
not		0
not		0
not	shalt	1
not	make	1

 Negative examples

Negative sampling

Pick randomly from vocabulary
(random sampling)

input word	output word	target
not	thou	1
not	aaron	0
not	taco	0
not	shalt	1
not	make	1

Word	Count	Probability
aardvark		
aarhus		
aaron		
taco		
thou		
zyzzyva		

La fórmula mágica de w2vec

Skipgram

shalt	not	make	a	machine
-------	-----	------	---	---------

input	output
make	shalt
make	not
make	a
make	machine

Negative Sampling

input word	output word	target
make	shalt	1
make	aaron	0
make	taco	0

Intuición sobre el proceso de training e inferencia en word2vec

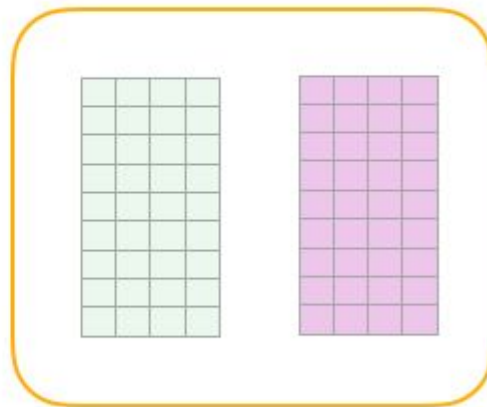


Parámetros a estimar (matrices)

dataset

input word	output word	target
not	thou	1
not	aaron	0
not	taco	0
not	shalt	1
not	mango	0
not	finglonger	0
not	make	1
not	plumbus	0
...	...	...

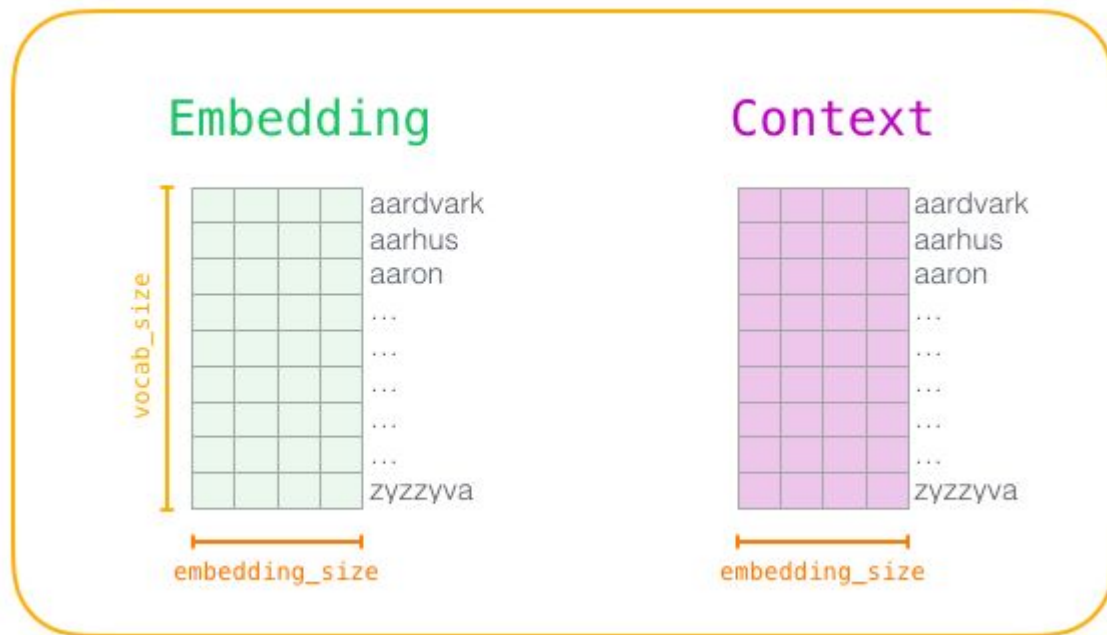
model



Parámetros a estimar (matrices)

dataset

input word	output word	target
not	thou	1
not	aaron	0
not	taco	0
not	shalt	1
not	mango	0
not	finglonger	0
not	make	1
not	plumbus	0
...	...	...



Veamos una iteración de training...

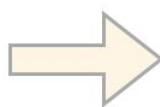
Embedding

				aardvark
				aarhus
				aaron
				...
				not
				...
				...
				...
				zyzzyva

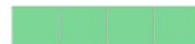
Context

				aardvark
				aarhus
				aaron
				...
				taco
				...
				thou
				...
				zyzzyva

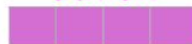
Look up
embeddings



not



aaron



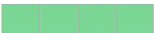
taco



thou



Veamos una iteración de training...

input word	output word	target	input • output
not 	thou 	1	0.2
not 	aaron 	0	-1.11
not 	taco 	0	0.74

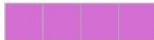
¿A a qué operación es equivalente la tercera columna?

Por ende... ¿qué está midiendo?

=> Este paso mide el dot.product de la input_word con cada término del contexto.

=> Mide la SIMILITUD entre cada par de vectores

Veamos una iteración de training...

input word	output word	target	input • output
not 	thou 	1	0.2
not 	aaron 	0	-1.11
not 	taco 	0	0.74

¿A a qué operación es equivalente la tercera columna?

Por ende... ¿qué está midiendo?

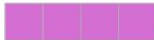
=> Este paso mide el dot.product de la input_word con cada término del contexto.

=> Mide la SIMILITUD entre cada par de vectores

Para ponerle una notación más linda:

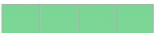
$$\text{Sim}(w,c) \approx w \cdot c$$

Veamos una iteración de training...

input word	output word	target	input • output	sigmoid()
not 	thou 	1	0.2	0.55
not 	aaron 	0	-1.11	0.25
not 	taco 	0	0.74	0.68



Veamos una iteración de training...

input word	output word	target	input • output	sigmoid()
not 	thou 	1	0.2	0.55
not 	aaron 	0	-1.11	0.25
not 	taco 	0	0.74	0.68

$$P(+|w, c) = \sigma(c \cdot w) = \frac{1}{1 + \exp(-c \cdot w)}$$

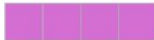
¿Cómo calcula SkipGram $P(+|w, c)$?

- Tenemos muchas palabras de contexto
- $\Rightarrow$ Asumimos independencia y las probabilidades se multiplican

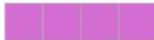
$$P(+|w, c_{1:L}) = \prod_{i=1}^L \sigma(c_i \cdot w)$$

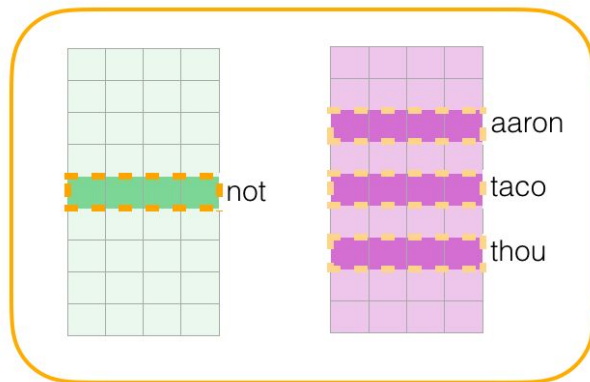
$$\log P(+|w, c_{1:L}) = \sum_{i=1}^L \log \sigma(c_i \cdot w)$$

Veamos una iteración de training...

input word	output word	target	input • output	sigmoid()	Error
not 	thou 	1	0.2	0.55	0.45
not 	aaron 	0	-1.11	0.25	-0.25
not 	taco 	0	0.74	0.68	-0.68

Veamos una iteración de training...

input word	output word	target	input • output	sigmoid()	Error
not 	thou 	1	0.2	0.55	0.45
not 	aaron 	0	-1.11	0.25	-0.25
not 	taco 	0	0.74	0.68	-0.68



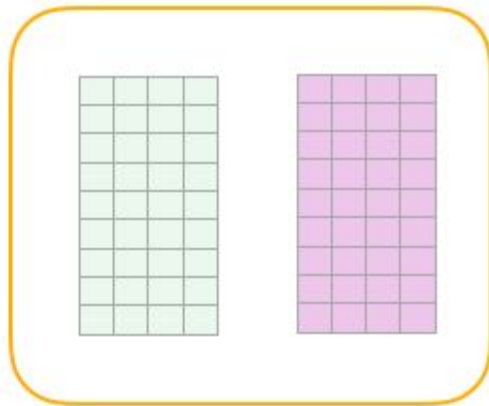
Update
Model
Parameters

Veamos una iteración de training...

dataset

input word	output word	target
not	thou	1
not	aaron	0
not	taco	0
not	shalt	1
not	mango	0
not	finglonger	0
not	make	1
not	plumbus	0
...	...	...

model

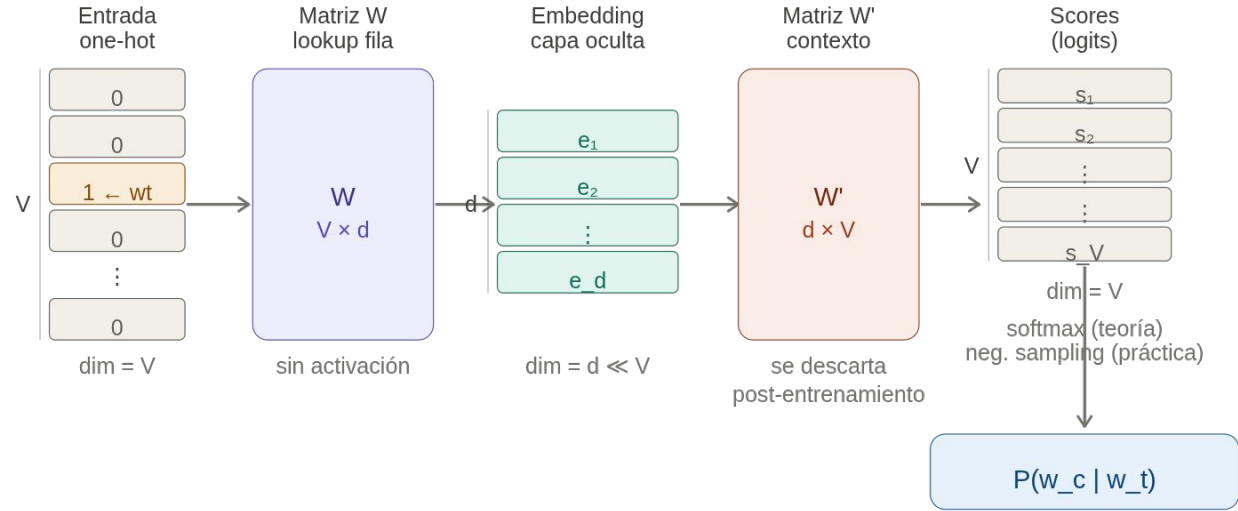


Un poquito más formalmente

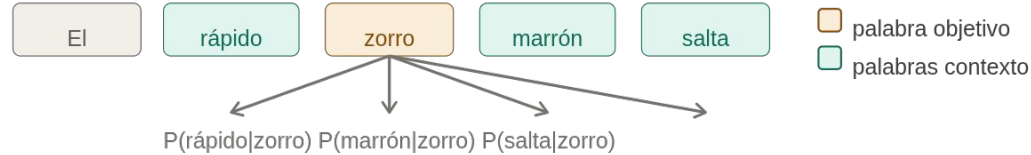
$$\begin{aligned} L_{CE} &= -\log \left[P(+|w, c_{pos}) \prod_{i=1}^k P(-|w, c_{neg_i}) \right] \\ &= - \left[\log P(+|w, c_{pos}) + \sum_{i=1}^k \log P(-|w, c_{neg_i}) \right] \\ &= - \left[\log P(+|w, c_{pos}) + \sum_{i=1}^k \log (1 - P(+|w, c_{neg_i})) \right] \\ &= - \left[\log \sigma(c_{pos} \cdot w) + \sum_{i=1}^k \log \sigma(-c_{neg_i} \cdot w) \right] \end{aligned}$$

- Datos
 - un conjunto de ejemplos positivos y negativos
 - un set inicial de embeddings (embedding + context)
- El objetivo del proceso de entrenamiento es ajustar esos vectores tal que
 - se maximice la similaridad (dot.product) entre las target_word y sus contextos positivos => (w, c_pos)
 - se minimice la similaridad entre los pares (w, c_neg)
- ¿Cómo? Stochastic Gradient Descent (queda para la materia de Deep Learning?)

Word2Vec — Skip-gram (arquitectura completa)



Pares de entrenamiento (ventana $w=2$)



Objetivo: maximizar $P(w_c | w_t)$ para pares reales — W resultante son los embeddings



Resumen de inferencia en word2vec

- Comienza con V vectores aleatorios de dimensión d como embeddings iniciales
- Entrena un clasificador basado en la similitud de embeddings:
 - Toma un corpus y pares de palabras que coocurren como ejemplos positivos
 - Toma pares de palabras que no coocurren como ejemplos negativos
 - Entrena el clasificador para distinguirlos ajustando lentamente todos los embeddings para mejorar el rendimiento del clasificador
 - Descarta el código del clasificador y conserva los embeddings



Otros métodos para construir embeddings

- word2vec fue pionero (2013) pero hoy hay métodos mejores
- GloVe: trabaja directamente sobre la matriz de co-ocurrencias

GloVe: Global Vectors for Word Representation

Jeffrey Pennington, Richard Socher, Christopher D. Manning

Introduction

GloVe is an unsupervised learning algorithm for obtaining vector representations for words. Training is performed on aggregated global word-word co-occurrence statistics from a corpus, and the resulting representations showcase interesting linear substructures of the word vector space.

Getting started (Code download)

- Download the latest [latest code](#) (licensed under the [Apache License, Version 2.0](#)). Look for "Clone or download"
- Unpack the files: `unzip master.zip`
- Compile the source: `cd GloVe-master && make`
- Run the demo script: `./demo.sh`
- Consult the included README for further usage details, or ask a [question](#)

Download pre-trained word vectors

- Pre-trained word vectors. This data is made available under the [Public Domain Dedication and License](#) v1.0 whose full text can be found at: <http://www.opendatacommons.org/licenses/pddl/1.0/>
 - [Wikipedia 2014 + Gigaword 5](#) (6B tokens, 400K vocab, uncased, 50d, 100d, 200d, & 300d vectors, 822 MB download): [glove.6B.zip](#)
 - Common Crawl (42B tokens, 19M vocab, uncased, 300d vectors, 175 GB download): [glove.42B.300d.zip](#)
 - Common Crawl (840B tokens, 2.2M vocab, cased, 300d vectors, 2.03 GB download): [glove.840B.300d.zip](#)
 - Twitter (2B tweets, 27B tokens, 1.2M vocab, uncased, 25d, 50d, 100d, & 200d vectors, 1.42 GB download): [glove.twitter.27B.zip](#)
- Ruby [script](#) for preprocessing Twitter data

Citing GloVe

Jeffrey Pennington, Richard Socher, and Christopher D. Manning. 2014. [GloVe: Global Vectors for Word Representation](#). [pdf] [bib]

Highlights

1. Nearest neighbors

The Euclidean distance (or cosine similarity) between two word vectors provides an effective method for measuring the linguistic or semantic similarity of the corresponding words. Sometimes, the nearest neighbors according to this metric reveal rare but relevant words that lie outside an average human's vocabulary. For example, here are the closest words to the target word *frog*:

0. *frog*
1. *frogs*
2. *toad*
3. *litoria*
4. *leptodactylidae*
5. *rana*
6. *lizard*
7. *eleutherodactylus*



3. litoria



4. leptodactylidae



5. rana



7. eleutherodactylus



Referencias

arXiv:1310.4546v1 [cs.CL] 16 Oct 2013

Distributed Representations of Words and Phrases and their Compositionality

Tomas Mikolov
Google Inc.
Mountain View
mikolov@google.com

Ilya Sutskever
Google Inc.
Mountain View
ilyasu@google.com

Kai Chen
Google Inc.
Mountain View
kai@google.com

Greg Corrado
Google Inc.
Mountain View
gcorrado@google.com

Jeffrey Dean
Google Inc.
Mountain View
jeff@google.com

Abstract

The recently introduced continuous Skip-gram model is an efficient method for learning high-quality distributed vector representations that capture a large number of precise syntactic and semantic word relationships. In this paper we present several extensions that improve both the quality of the vectors and the training speed. By subsampling of the frequent words we obtain significant speedup and also learn more regular word representations. We also describe a simple alternative to the hierarchical softmax called negative sampling.

An inherent limitation of word representations is their indifference to word order and their inability to represent idiomatic phrases. For example, the meanings of "Canada" and "Air" cannot be easily combined to obtain "Air Canada". Motivated by this example, we present a simple method for finding phrases in text, and show that learning good vector representations for millions of phrases is possible.

1 Introduction

Distributed representations of words in a vector space help learning algorithms to achieve better performance in natural language processing tasks by grouping similar words. One of the earliest use of word representations dates back to 1986 due to Rumelhart, Hinton, and Williams [13]. This idea has since been applied to statistical language modeling with considerable success [1]. The follow up work includes applications to automatic speech recognition and machine translation [14, 7], and a wide range of NLP tasks [2, 20, 15, 3, 18, 19, 9].

Recently, Mikolov et al. [8] introduced the Skip-gram model, an efficient method for learning high-quality vector representations of words from large amounts of unstructured text data. Unlike most of the previously used neural network architectures for learning word vectors, training of the Skip-gram model (see Figure 1) does not involve dense matrix multiplications. This makes the training extremely efficient: an optimized single-machine implementation can train on more than 100 billion words in one day.

The word representations computed using neural networks are very interesting because the learned vectors explicitly encode many linguistic regularities and patterns. Somewhat surprisingly, many of these patterns can be represented as linear translations. For example, the result of a vector calculation $\text{vec}(\text{"Madrid"}) - \text{vec}(\text{"Spain"}) + \text{vec}(\text{"France"})$ is closer to $\text{vec}(\text{"Paris"})$ than to any other word vector [9, 8].

arXiv:1301.3781v3 [cs.CL] 7 Sep 2013

Efficient Estimation of Word Representations in Vector Space

Tomas Mikolov
Google Inc., Mountain View, CA
tmikolov@google.com

Kai Chen
Google Inc., Mountain View, CA
kaichen@google.com

Greg Corrado
Google Inc., Mountain View, CA
gcorrado@google.com

Jeffrey Dean
Google Inc., Mountain View, CA
jeff@google.com

Abstract

We propose two novel model architectures for computing continuous vector representations of words from very large data sets. The quality of these representations is measured in a word similarity task, and the results are compared to the previously best performing techniques based on different types of neural networks. We observe large improvements in accuracy at much lower computational cost, i.e. it takes less than a day to learn high quality word vectors from a 1.6 billion words data set. Furthermore, we show that these vectors provide state-of-the-art performance on our test set for measuring syntactic and semantic word similarities.

1 Introduction

Many current NLP systems and techniques treat words as atomic units - there is no notion of similarity between words, as these are represented as indices in a vocabulary. This choice has several good reasons - simplicity, robustness and the observation that simple models trained on huge amounts of data outperform complex systems trained on less data. An example is the popular N-gram model used for statistical language modeling - today, it is possible to train N-grams on virtually all available data (trillions of words [3]).

However, the simple techniques are at their limits in many tasks. For example, the amount of relevant in-domain data for automatic speech recognition is limited - the performance is usually dominated by the size of high quality transcribed speech data (often just millions of words). In machine translation, the existing corpora for many languages contain only a few billions of words or less. Thus, there are situations where simple scaling up of the basic techniques will not result in any significant progress, and we have to focus on more advanced techniques.

With progress of machine learning techniques in recent years, it has become possible to train more complex models on much larger data set, and they typically outperform the simple models. Probably the most successful concept is to use distributed representations of words [10]. For example, neural network based language models significantly outperform N-gram models [1, 27, 17].

1.1 Goals of the Paper

The main goal of this paper is to introduce techniques that can be used for learning high-quality word vectors from huge data sets with billions of words, and with millions of words in the vocabulary. As far as we know, none of the previously proposed architectures has been successfully trained on more

